

ELEC6910J

Deep Reinforcement Learning

Lecture 3: Multi-Armed Bandits

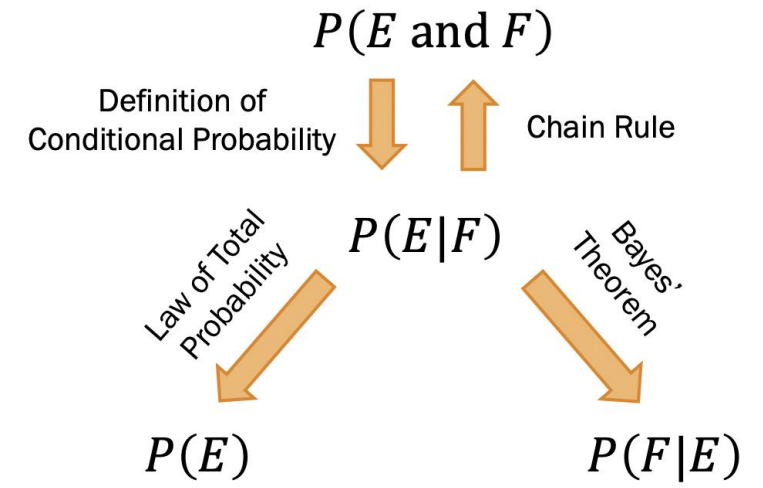
Course Instructor: Ling PAN

Department of Electronic and Computer Engineering

Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188, CS285, CMU 10-403

Recap: The Core Probability Toolkit



Definition of Conditional Probability

$$P(E|F) = \frac{P(E \text{ and } F)}{P(F)}$$

The Law of Total Probability

$$P(E) = P(E \text{ and } F) + P(E \text{ and } F^C)$$

$$P(E) = \sum_{i=1}^n P(E \text{ and } B_i)$$

$$P(E) = P(E|F)P(F) + P(E|F^C)P(F^C)$$

$$= \sum_{i=1}^n P(E|B_i)P(B_i)$$

Bayes' Theorem

$$P(B|E) = \frac{P(E|B) \cdot P(B)}{P(E)}$$

$$P(B|E) = \frac{P(E|B) \cdot P(B)}{P(E|B) \cdot P(B) + P(E|B^C) \cdot P(B^C)}$$

Independence

$$P(E|F) = P(E)$$

$$P(E \text{ and } F) = P(E)P(F)$$

Axiom 1: $0 \leq P(E) \leq 1$

Axiom 2: $P(S) = 1$

Axiom 3: If E and F are mutually exclusive, then $P(E \text{ or } F) = P(E) + P(F)$

Otherwise, use Inclusion-Exclusion:

$$P(E \text{ or } F) = P(E) + P(F) - P(E \text{ and } F)$$

$$P(E^C) = 1 - P(E)$$

Chain Rule

$$\begin{aligned} P(E \text{ and } F) &= P(E|F) \cdot P(F) \\ &= P(F|E) \cdot P(E) \end{aligned}$$

De Morgan's Laws

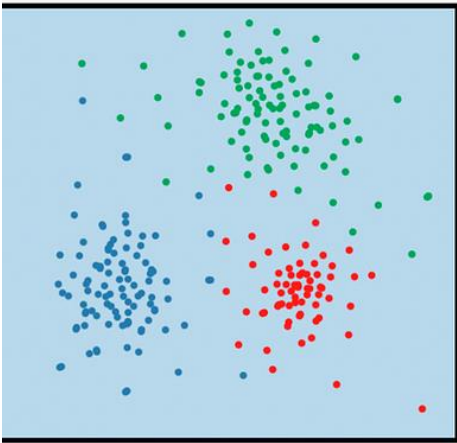
$$(A \text{ or } B)^C = A^C \text{ and } B^C$$

$$(A \text{ and } B)^C = A^C \text{ or } B^C$$

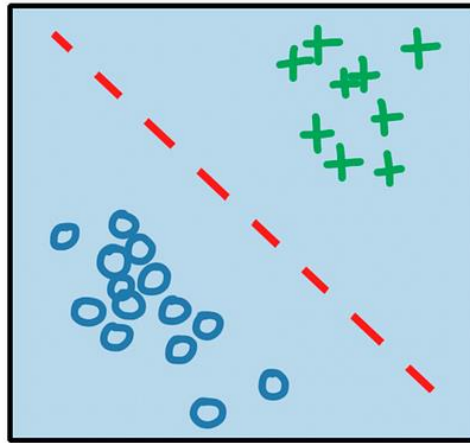
Recap: Machine Learning

machine learning

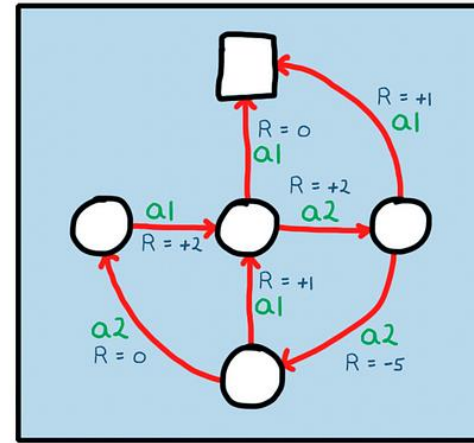
unsupervised
learning



supervised
learning

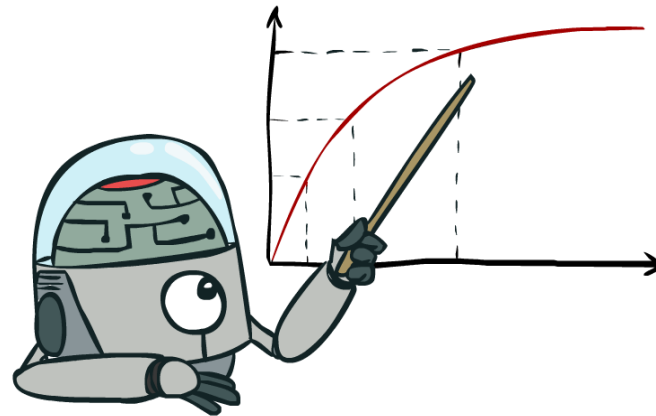


reinforcement
learning



Recap

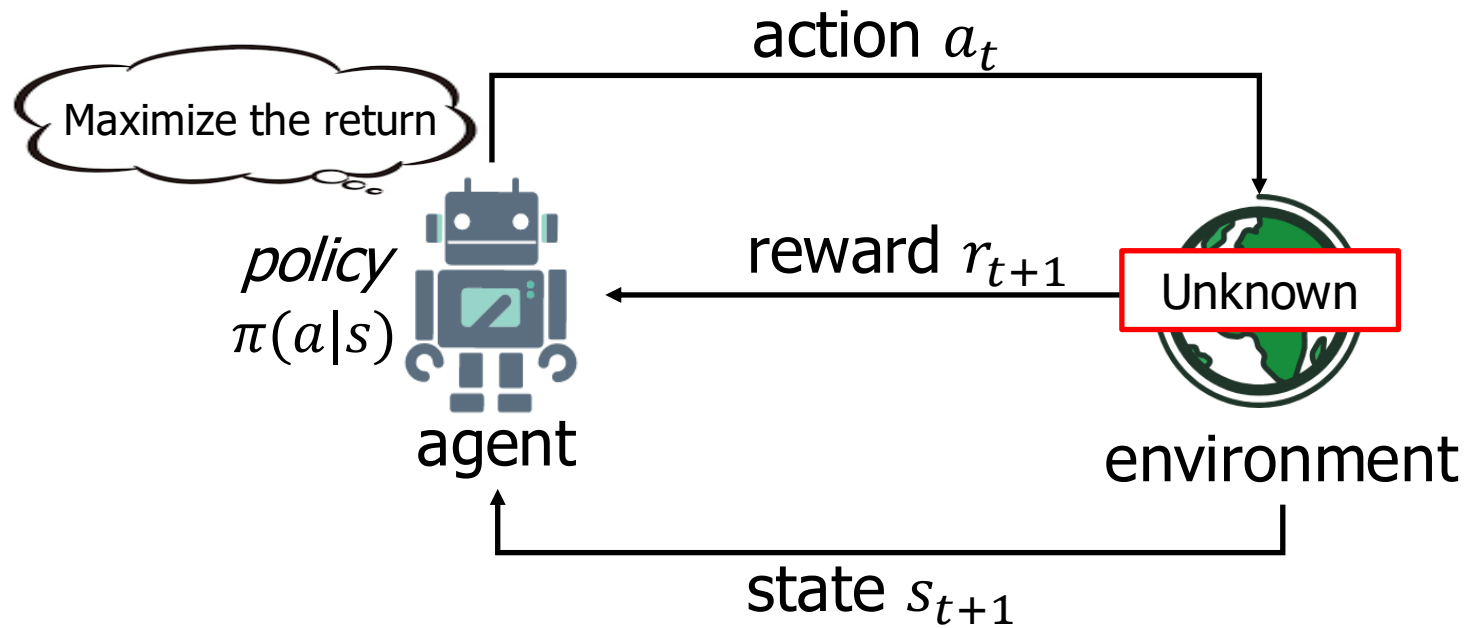
Maximize Your Expected Utility



→ Reinforcement
Learning

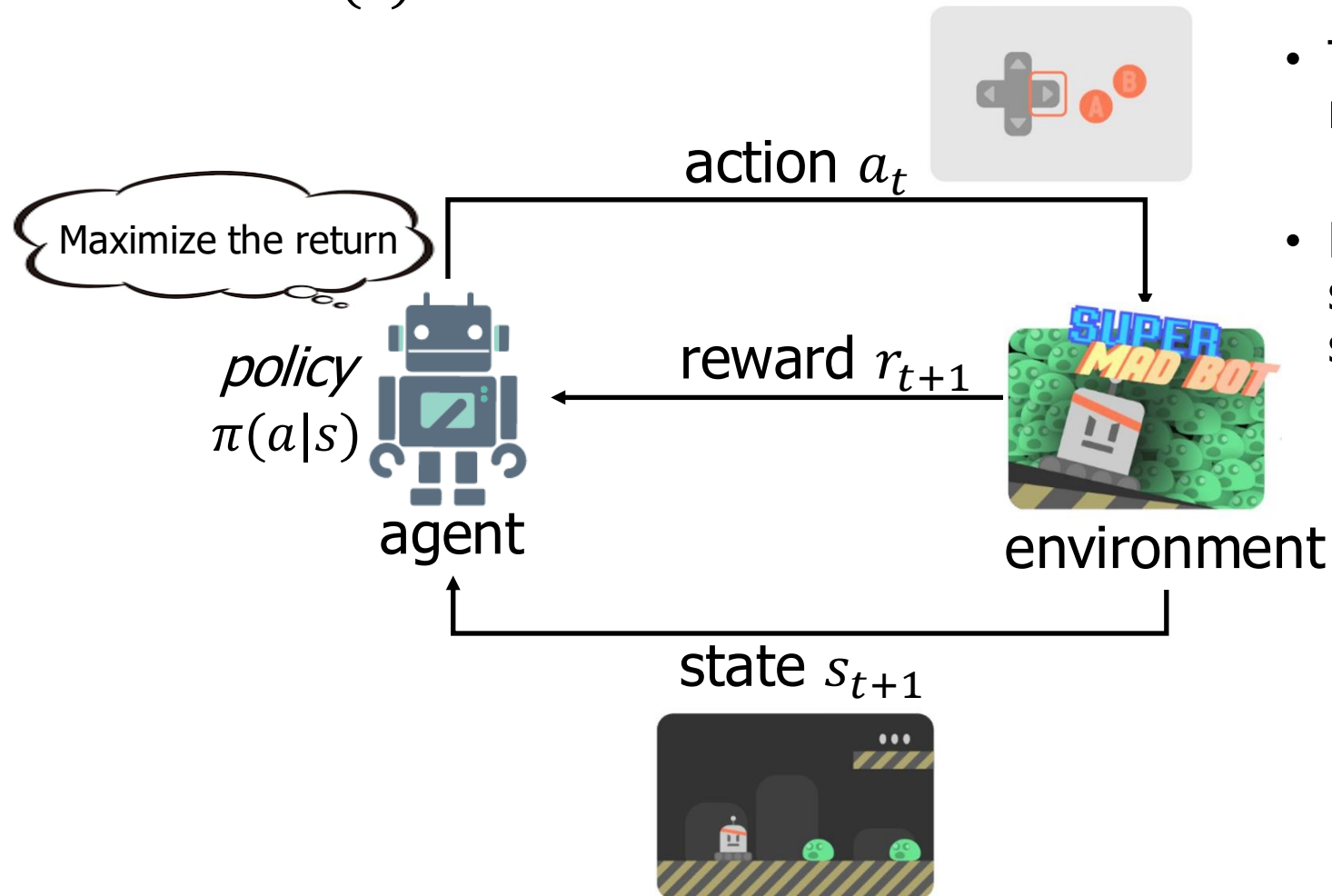
Reinforcement Learning

Goal: learn a policy π to maximize the expected long-term rewards by interacting with the environment



Reinforcement Learning

Given an initial state distribution $\mu_0(s_0)$, estimate parameters θ of a policy π_θ so that, the trajectories τ sampled from this policy have maximum returns, i.e., sum of rewards $R(\tau)$



- Trajectory τ : a sequence of state, action, rewards, a game fragment or a full game
 $\tau: S_0, a_0, r_0, S_1, a_1, r_1, \dots, S_T, a_T, r_T$
- Reward of a trajectory $R(\tau)$: (discounted) sum of the rewards of the individual state/actions

$$R(\tau) = \sum_{i=1}^T r_t$$

Multi-Armed Bandits

- Motivation: Learning to act in non-sequential (single action) setups
 - Each action results in an immediate reward
 - We want to choose actions that maximize our immediate reward in expectation



One-armed bandit
(A single slot machine)



Multi-armed bandit
(Multiple slot machines)

$a_t, r_{t+1}, a_{t+1}, r_{t+2}, \dots$
The state does not change
(stateless)

Multi-Armed Bandits

Setup

Definitions:

- One-armed bandit: a single slot machine
- Multi-armed bandit: a row of multiple slot machines (K arms)
- Goal: maximize cumulative reward over time



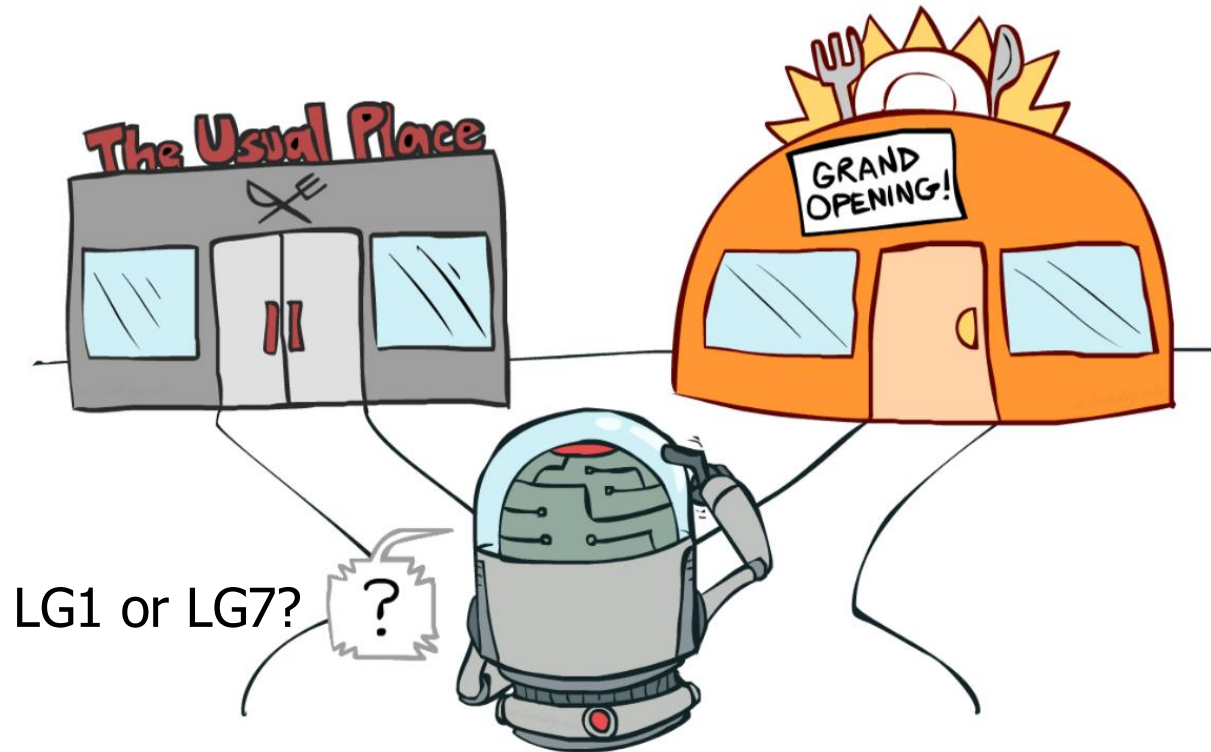
- You have K arms (machines) to pull.
- At each timestep t , you choose one arm a_t and get a reward r_t

Multi-Armed Bandits

- Example 1: displaying an advertisement can generate or not a click by a user in different points in time, depending on the weather, the user's mood, etc.
 - Actions: which advertisement to display
 - Reward (binary): whether the user clicks on the displayed ad or not

Multi-Armed Bandits

- Example 2: choosing which restaurant to go to
 - Actions: the restaurants to choose from
 - Rewards: your pleasure



Multi-Armed Bandits

- Example 2: choosing which restaurant to go to
 - Actions: the restaurants to choose from
 - Rewards: your pleasure
- Question: What if you become friends with the restaurant employees and enjoy interacting with them?
- This is not a stateless setup! If the state changes as a result of your actions, then we have a sequential setup, and we will address it in the next lecture.

Multi-Armed Bandits

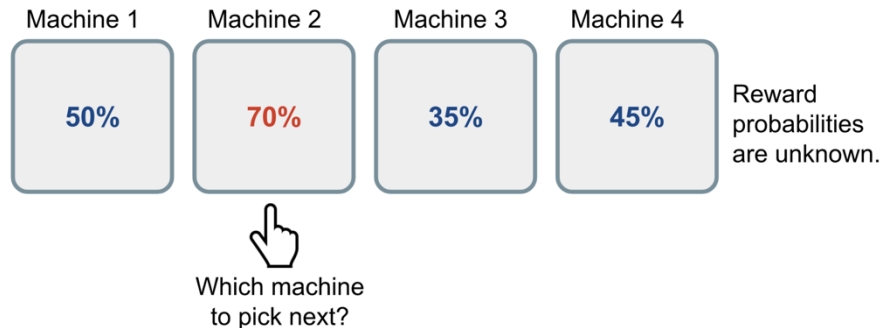
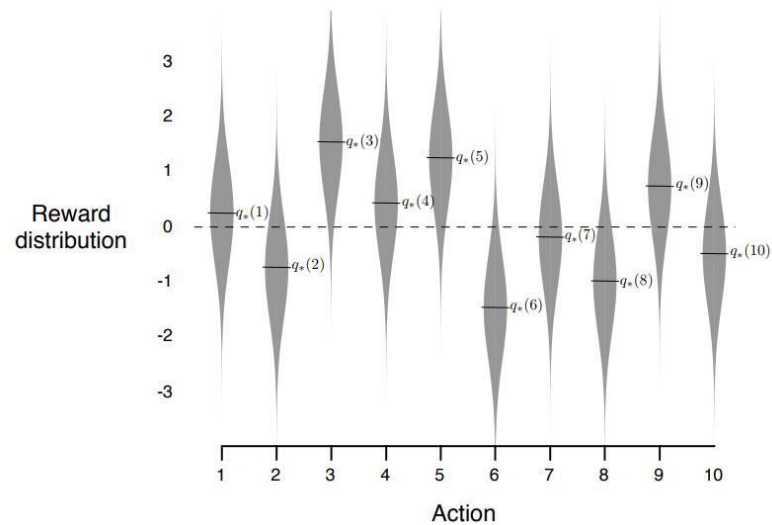
- At each timestep t the agent chooses one of the K arms and plays it.
- The k -th arm produces reward $r_{k,t}$ when played at timestep t .
- Agent's objective: maximize cumulative rewards (over a finite or infinite horizon).
- Challenge: The agent does not know neither the reward distributions neither their means.

Multi-Armed Bandits

- The rewards $r_{k,t}$ are drawn from a probability distribution P_k with mean μ_k .
- Agent's objective: maximize cumulative rewards (over a finite or infinite horizon).
 - Optimal strategy: I can maximize cumulative rewards over a finite or infinite horizon if I just play the arm with the highest mean reward μ_k each time (but I do not know which one it is)

Multi-Armed Bandits

- How can we make decisions?
- Action-value
 - Definition: the action-value for action a is its mean reward $q_*(a) = \mathbb{E}[R_t | A_t = a]$



Multi-Armed Bandits



- How can we make decisions?
- Action-value
 - Definition: the action-value for action a is its mean reward $q_*(a) = \mathbb{E}[R_t | A_t = a]$
- Suppose you form estimates

$$Q_t(a) \approx q_*(a), \forall a \text{ action-value estimates}$$

- Define the greedy at time t as

$$A_t^* = \operatorname{argmax}_a Q_t(a)$$

- If $A_t = A_t^*$, then you are exploiting
- If $A_t \neq A_t^*$, then you are exploring
- You can't do both, but you need to do both
- You can never stop exploring, but maybe you should explore less with time

The Exploration/Exploitation Dilemma

- Suppose you form estimates

$$Q_t(a) \approx q_*(a), \forall a \text{ action-value estimates}$$

- Define the greedy at time t as

$$A_t^* = \operatorname{argmax}_a Q_t(a)$$

- If $A_t = A_t^*$, then you are exploiting
- If $A_t \neq A_t^*$, then you are exploring
- You can't do both, but you need to do both
- You can never stop exploring, but maybe you should explore less with time

The Exploration/Exploitation Dilemma

- Online decision-making involves a fundamental choice:
 - Exploitation: make the best decision given current information
 - Exploration: gather more information
- The best long-term strategy may involve short-term sacrifices
- Gather enough information to make the best overall decisions
- The exploration/exploitation dilemma is not an only problem encountered in computational RL or deep RL: it is a fundamental problem in decision making of any intelligent agent

The Exploration/Exploitation Dilemma

- Example 1: Restaurant Selection
 - Exploitation: Go to your favorite restaurant
 - Exploration: Try a new restaurant
- Example 2: Oil Drilling
 - Exploitation: Drill at the best known location
 - Exploration: Drill at a new location
- Example 3: Game Playing
 - Exploitation: Play the move you believe is best
 - Exploration: Play an experimental move



Regret

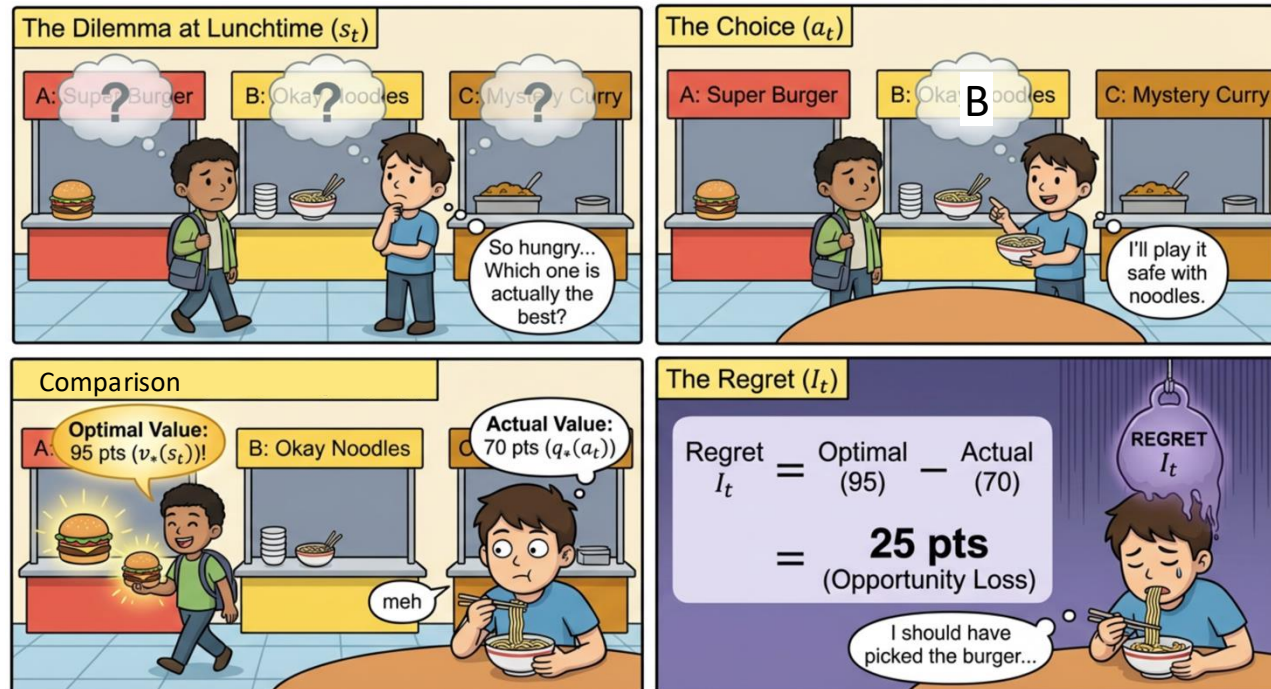
- The action-value is the mean reward for action a

$$q_*(a) = \mathbb{E}[R_t | A_t = a], \forall a \in \{1, \dots, k\}$$

- The optimal value is

$$v_* = q(a^*) = \max_{a \in \mathcal{A}} q_*(a)$$

- Example



Regret

- The action-value is the mean reward for action a

$$q_*(a) = \mathbb{E}[R_t | A_t = a], \forall a \in \{1, \dots, k\}$$

- The optimal value is

$$v_* = q(a^*) = \max_{a \in \mathcal{A}} q_*(a)$$

- The regret is the expected opportunity loss for one step. For an algorithm that selects action a_t at timestep t it reads:

$$I_t = \mathbb{E}[v_* - q_*(a_t)]$$

Forming Action-Value Estimates

- To simplify notation, let us focus on one action
 - Its action value estimate after $n - 1$ rewards

$$Q_n = \frac{R_1 + R_2 + \dots + R_{n-1}}{n - 1}$$

Sample average
(for stationary bandits)

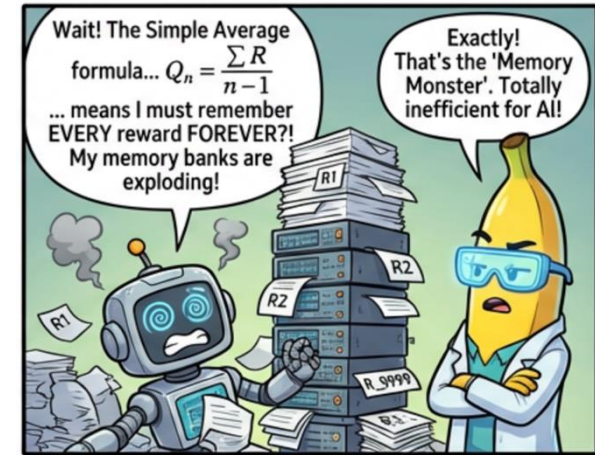


Does it incur any computation/memory challenge?

Forming Action-Value Estimates

- To simplify notation, let us focus on one action
 - Its action value estimate after $n - 1$ rewards

$$Q_n = \frac{R_1 + R_2 + \dots + R_{n-1}}{n - 1}$$



- Q_n : our current estimate of how good an action is after trying it $n-1$ times.
 - Sum up every single reward we have ever gotten and divide by the count.
 - Problem:
 - Imagine you are an AlphaGo agent playing your 10-millionth game of Go.
 - To calculate your estimate for a move using this formula, you would need to have stored the specific rewards from the previous 9,999,999 games in memory, fetch them all, add them up, and divide every single step.
- Inefficient! It eats up memory and slows down computation.
- We need a way to update our knowledge without needing a perfect memory of the entire past.
 - How can we do this incrementally (without storing all the rewards)?

Forming Action-Value Estimates

- To simplify notation, let us focus on one action

- Its action value estimate after $n - 1$ rewards

$$Q_n = \frac{R_1 + R_2 + \cdots + R_{n-1}}{n - 1}$$

Sample average
(for stationary bandits)

- How can we do this incrementally (without storing all the rewards)?
 - Can store a running sum, or equivalently:

$$Q_{n+1} = Q_n + \frac{1}{n} [R_n - Q_n]$$

- Why?

Forming Action-Value Estimates

- To simplify notation, let us focus on one action
 - Derivation:

$$\begin{aligned}Q_n &\doteq \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1} \\Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\&= \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) \\&= \frac{1}{n} \left(R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i \right) \\&= \frac{1}{n} \left(R_n + (n-1) Q_n \right) \\&= \frac{1}{n} \left(R_n + n Q_n - Q_n \right) \\&= Q_n + \frac{1}{n} \left[R_n - Q_n \right],\end{aligned}$$

Forming Action-Value Estimates

- To simplify notation, let us focus on one action

- Its action value estimate after $n - 1$ rewards

$$Q_n = \frac{R_1 + R_2 + \cdots + R_{n-1}}{n - 1}$$

- How can we do this incrementally (without storing all the rewards)?
 - Can store a running sum, or equivalently:

$$Q_{n+1} = Q_n + \frac{1}{n} [R_n - Q_n]$$

- This implementation requires memory only for Q_k and k , and only the small computation for each new reward.

Forming Action-Value Estimates

$$Q_{n+1} = Q_n + \frac{1}{n} [R_n - Q_n]$$

- If $R_n > Q_n$, reality is better than your expectation (Surprise! 🤩), so you should increase your estimate.
- If $R_n < Q_n$, reality is worse than your expectation (Disappointment... 😞), so you should decrease your estimate.
- This difference is called the Error.
- What happens as n gets bigger? → The step size gets smaller.
- Example: If you have tried a restaurant 100 times (high n), your estimate of it is probably pretty solid. One weird meal should not change your opinion too much. So, the step size $\frac{1}{100}$ is small. We trust our old estimate more.
→ We will call this the learning rate.

- Generalization: This is a standard form for learning/update rules throughout this course:

$$\text{New Estimate} \leftarrow \text{Old Estimate} + \text{Step Size} \times \underbrace{[\text{Target} - \text{Old Estimate}]}_{\text{Error}}$$

Action selection in multi-armed bandits

- Fixed exploration period + Greedy

1. Allocate a fixed time period to exploration when you try bandits uniformly at random
2. Estimate mean rewards for all actions: $Q_t(a) = \frac{1}{N_t(a)} \sum_{i=1}^{t-1} r_i \mathbb{I}(A_i = a)$
3. Select the action that is optimal for the estimated mean rewards (breaking ties at random):
$$a_t = \operatorname{argmax}_a Q_t(a)$$
4. GOTO 2

Epsilon-Greedy Action Selection

- In greedy action selection, you always exploit
- In ϵ -greedy, you are usually greedy, but with probability ϵ you instead pick an action at random (possibly the greedy action again)
- This is perhaps the simplest way to balance exploration and exploitation

Epsilon-Greedy Action Selection

- In greedy action selection, you always exploit
- In ϵ -greedy, you are usually greedy, but with probability ϵ you instead pick an action at random (possibly the greedy action again)
- This is perhaps the simplest way to balance exploration and exploitation

A simple bandit algorithm

Initialize, for $a = 1$ to k :

$$Q(a) \leftarrow 0$$

$$N(a) \leftarrow 0$$

Repeat forever:

$$A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{with probability } 1 - \epsilon \quad (\text{breaking ties randomly}) \\ \text{a random action} & \text{with probability } \epsilon \end{cases}$$

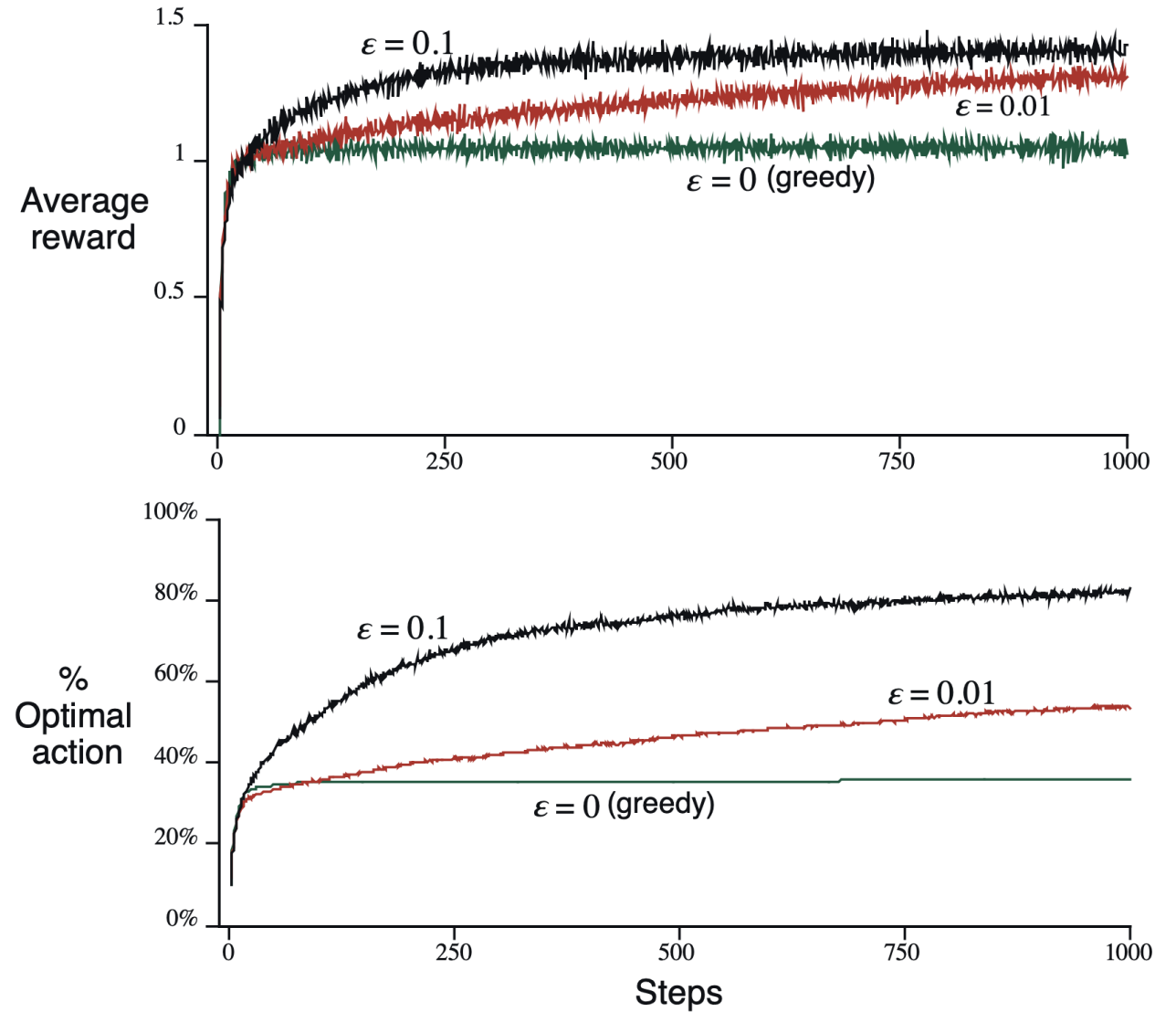
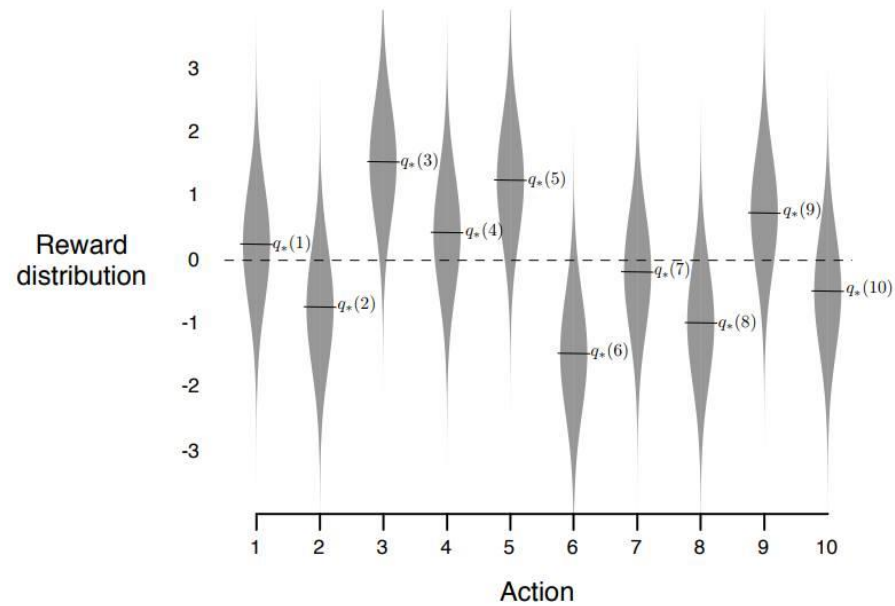
$$R \leftarrow \text{bandit}(A)$$

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$$

- The ϵ -greedy algorithm continues to explore forever
 - With probability $1 - \epsilon$ select $a_t = \arg \max_a Q_t(a)$
 - With probability ϵ select a random action (independent of its Q-value estimate)

Greedy v.s. Epsilon-Greedy



Non-Stationary Bandits

- Suppose the true action values change slowly over time
 - Then we say that the problem is nonstationary
 - In this case, sample averages are not a good idea
- Better is an “exponential, recency-weighted average”: weight recent rewards more heavily than long-past ones
 - A popular way: use a constant step-size parameter $\alpha \in (0,1]$, i.e., $Q_{k+1} = Q_k + \alpha[R_k - Q_k]$

$$\begin{aligned} Q_{k+1} &= Q_k + \alpha[R_k - Q_k] \\ &= \alpha R_k + (1 - \alpha)Q_k \quad (\text{combine common terms}) \\ &= \alpha R_k + (1 - \alpha)[\alpha R_{k-1} + (1 - \alpha)Q_{k-1}] \quad (\text{expand } Q_k) \\ &= \alpha R_k + (1 - \alpha)\alpha R_{k-1} + (1 - \alpha)^2 Q_{k-1} \\ &= \alpha R_k + (1 - \alpha)\alpha R_{k-1} + (1 - \alpha)^2 \alpha R_{k-2} + \\ &\quad \dots + (1 - \alpha)^{k-1} \alpha R_1 + (1 - \alpha)^k Q_1 \\ &= (1 - \alpha)^k Q_1 + \sum_{i=1}^k \alpha (1 - \alpha)^{k-i} R_i. \end{aligned}$$

Non-Stationary Bandits

- Suppose the true action values change slowly over time
 - Then we say that the problem is nonstationary
 - In this case, sample averages are not a good idea
- Better is an “exponential, recency-weighted average”: weight recent rewards more heavily than long-past ones
 - A popular way: use a constant step-size parameter $\alpha \in (0,1]$, i.e., $Q_{k+1} = Q_k + \alpha[R_k - Q_k]$
Result: a weighted average of past rewards and the initial estimate Q_1

$$Q_{k+1} = (1 - \alpha)^k Q_1 + \sum_{i=1}^k \alpha(1 - \alpha)^{k-i} R_i, \text{ where } (1 - \alpha)^k + \sum_{i=1}^k \alpha(1 - \alpha)^{k-i} = 1$$

- The smaller the $i \rightarrow$ the smaller the term $(1 - \alpha)^{k-i} \rightarrow$ forget earlier rewards
- The weight decays exponentially according to the exponent on $1 - \alpha$
- Can also consider varying the step-size parameter from step to step
 - Convergence may not be guaranteed

Action selection in multi-armed bandits

- Fixed exploration period + Greedy

1. Allocate a fixed time period to exploration when you try bandits uniformly at random
2. Estimate mean rewards for all actions: $Q_t(a) = \frac{1}{N_t(a)} \sum_{i=1}^{t-1} r_i \mathbb{I}(A_i = a)$
3. Select the action that is optimal for the estimated mean rewards (breaking ties at random):
$$a_t = \operatorname{argmax}_a Q_t(a)$$
4. GOTO 2

Contextual Bandits

- A contextual bandit is a tuple (A, S, R)
 - A is a known set of k actions (or “arms”)
 - $S = \mathbb{P}[s]$ is an unknown distribution over states (or “contexts”)
 - $R_s^a(r)$ is an unknown probability distribution over rewards
 - At each time t
 - Environment generates state $s_t \sim S$
 - Agent selects action $a_t \in A$
 - Environment generates reward $r \sim R_s^a(r)$
 - The goal is to maximize cumulative reward $\sum_{i=1}^t r_t$
- Example: which lever you pull determines the next machine you enter

Multi-Armed Bandits

- Previous example: choosing which restaurant to go to
 - Actions: the restaurants to choose from
 - Rewards: your pleasure
- Question: What if you become friends with the restaurant employees and enjoy interacting with them?
- This is not a stateless setup! If the state changes as a result of your actions, then we have a sequential setup, and we will address it in the next lecture.

ELEC6910J

Deep Reinforcement Learning

Lecture 4: Markov Decision Processes

Course Instructor: Ling PAN

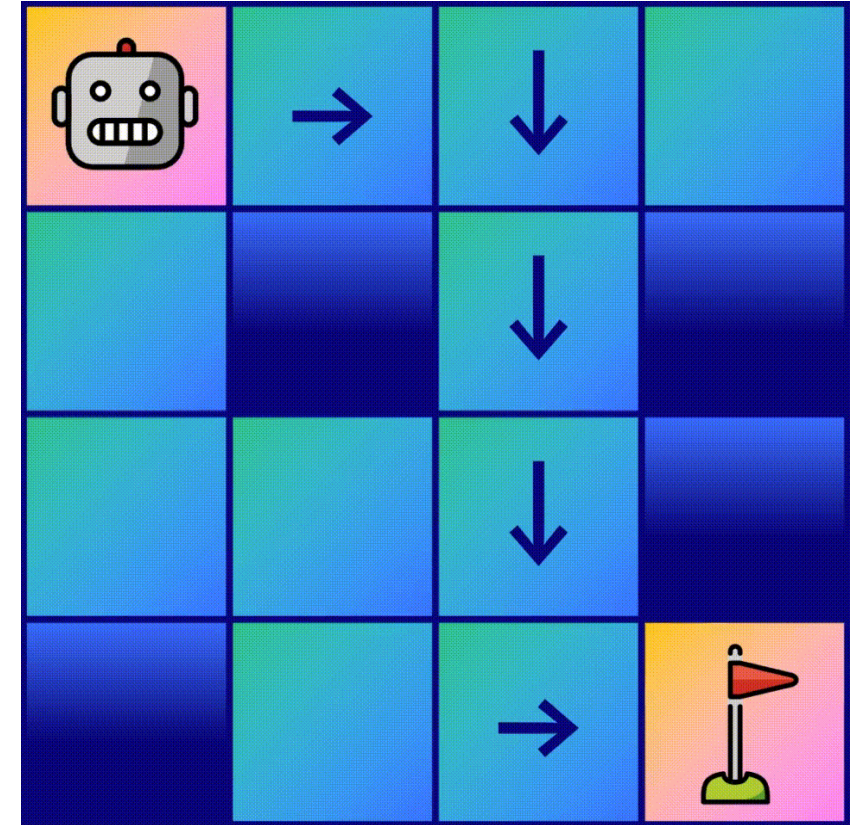
Department of Electronic and Computer Engineering

Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188, CS286, CMU 10-403

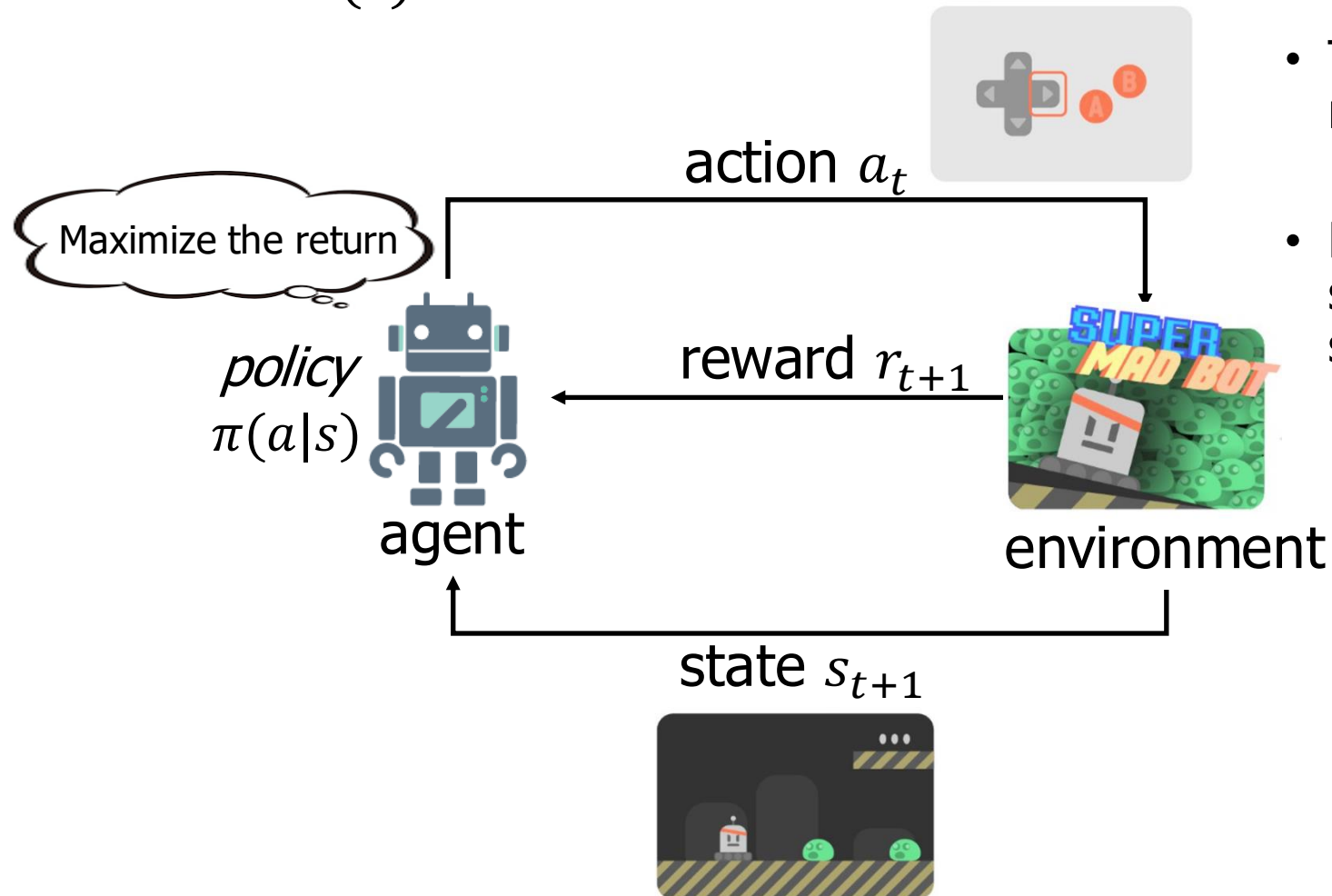
Definitions

- Agent
 - An entity that is equipped with sensors, in order to sense the environment, and end-effectors in order to act in the environment, and goals that he wants to achieve
- Policy
 - A mapping function from observations (sensations, inputs of the sensors) to actions of the end effectors.
- Model
 - The mapping function from states/observations and actions to future states/observations
- Planning:
 - (many definitions but one of them is) unrolling a model forward in time and selecting the best action sequence that satisfies a specific goal
- Plan:
 - A sequence of actions



Reinforcement Learning

Given an initial state distribution $\mu_0(s_0)$, estimate parameters θ of a policy π_θ so that, the trajectories τ sampled from this policy have maximum returns, i.e., sum of rewards $R(\tau)$



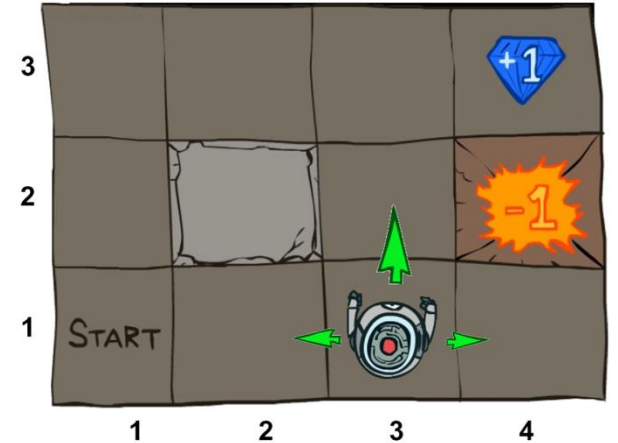
- Trajectory τ : a sequence of state, action, rewards, a game fragment or a full game
 $\tau: s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T$
- Reward of a trajectory $R(\tau)$: (discounted) sum of the rewards of the individual state/actions

$$R(\tau) = \sum_{i=1}^T r_t$$

Markov Decision Process (MDP)

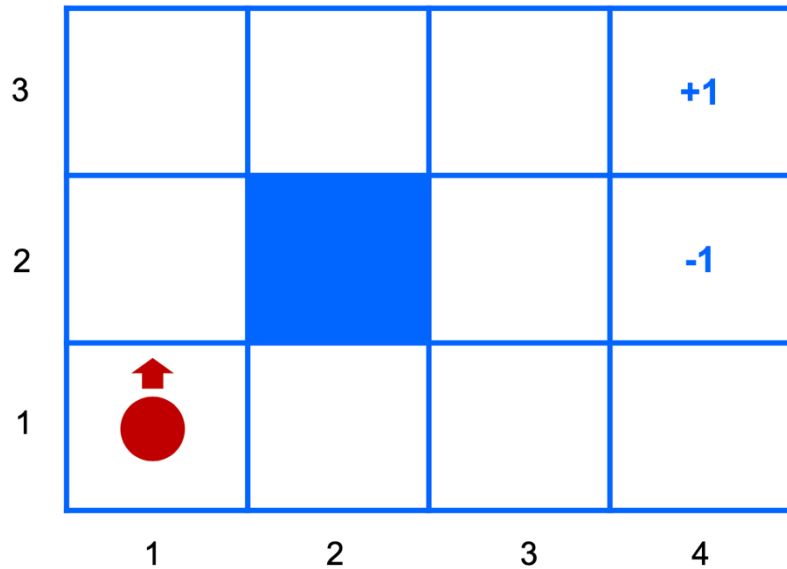
- Example: GridWorld

- A maze-like problem
 - The agent lives in a grid
 - Walls block the agent's path
- Noisy movement: actions do not always go as planned
 - 80% of the time, the action North takes the agent North (if there is no wall there)
 - 10% of the time, North takes the agent West;
 - 10% East
 - If there is a wall in the direction the agent would have been taken, the agent stays put
- The agent receives rewards each time step
 - Small "living" reward each step (can be negative)
 - Big rewards come at the end (good or bad)
- Goal: maximize sum of rewards



Markov Decision Process (MDP)

- Example: GridWorld



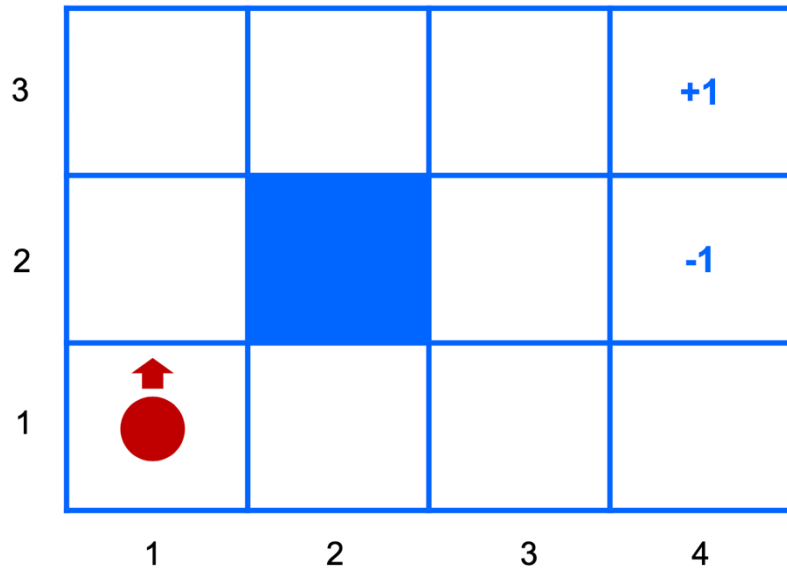
s	a	s'	R
(1,1)	north	?	?

Noisy movement: actions do not always go as planned

- 80% of the time, the action North takes the agent North (if there is no wall there)
- 10% of the time, North takes the agent West;
- 10% East

Markov Decision Process (MDP)

- Example: GridWorld



s	a	s'	R
(1,1)	north	?	?

Noisy movement: actions do not always go as planned

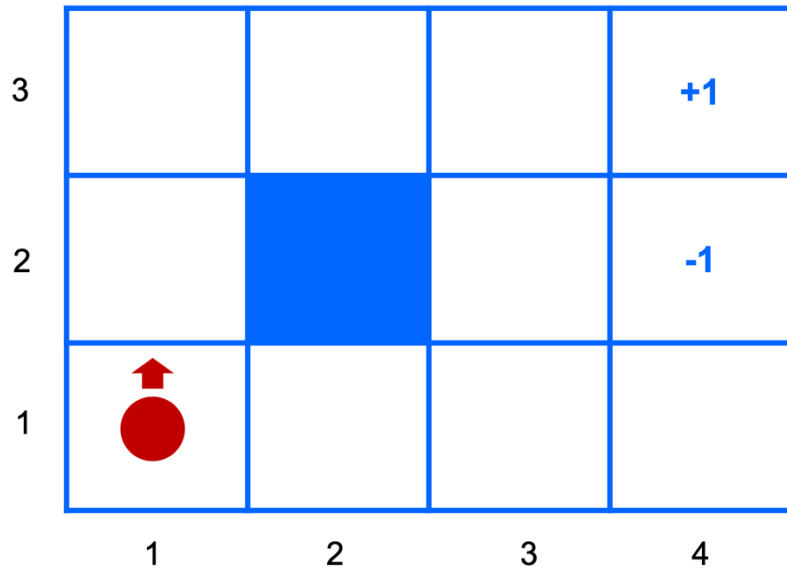
- 80% of the time, the action North takes the agent North (if there is no wall there)
- 10% of the time, North takes the agent West;
- 10% East

Transition dynamics $T(s, a, s')$:

- $T((1,1), \text{north}, (2,1)) = 0.8$
- $T((1,1), \text{north}, (1,2)) = 0.1$
- $T((1,1), \text{north}, (1,1)) = 0.1$

Markov Decision Process (MDP)

- Example: GridWorld



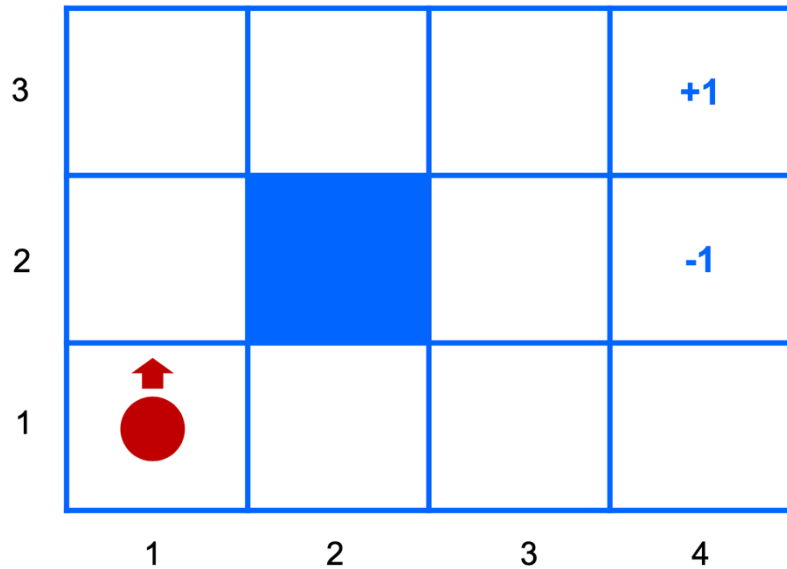
s	a	s'	R
(1,1)	north	?	?

The agent receives rewards each time step

- Small “living” reward each step (can be negative)
- Big rewards come at the end (good or bad)

Markov Decision Process (MDP)

- Example: GridWorld



s	a	s'	R
(1,1)	north	?	?

The agent receives rewards each time step

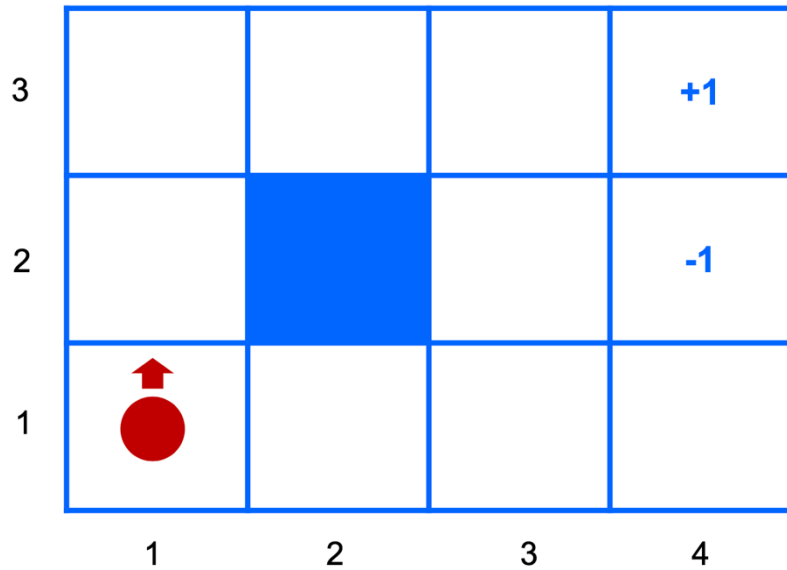
- Small “living” reward each step (can be negative)
- Big rewards come at the end (good or bad)

Reward function $R(s, a, s')$:

- $R((1,1), \text{north}, (2,1)) = -0.1$

Markov Decision Process (MDP)

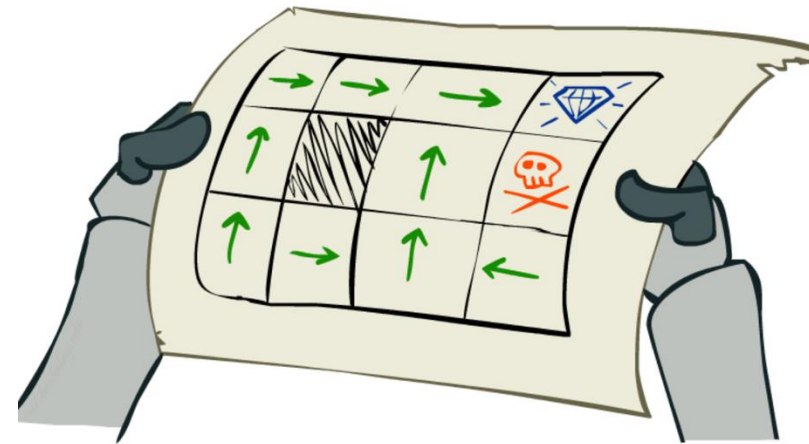
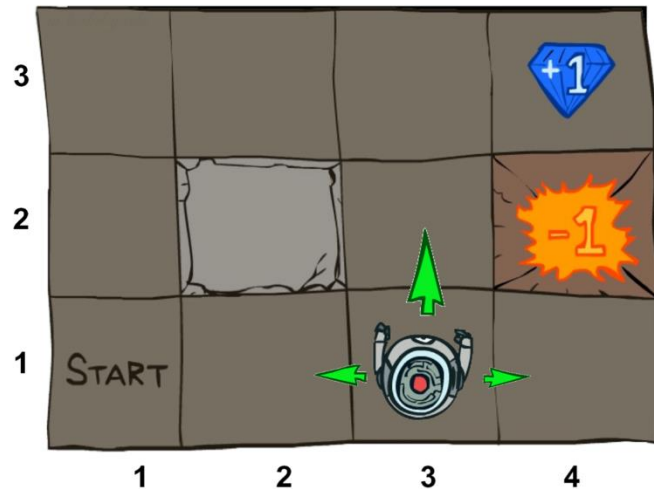
- Example: GridWorld



s	a	s'	R
(1,1)	north	(2,1)	-0.1
(2,1)	north	(3,1)	-0.1
(3,1)	east	(2,1)	-0.1
(2,1)	north	(3,1)	-0.1
(3,1)	east	(2,3)	-0.1
(2,3)	east	(3,3)	-0.1
(3,3)	east	(4,3)	-0.1
(4,3)	exit	(4,3)	+1.0

Policies

- In deterministic single-agent search problems, we wanted an optimal plan, or sequence of actions, from start to a goal
- For MDPs, we want an optimal policy $\pi^*: S \rightarrow A$
 - A policy π gives an action for each state
 - An optimal policy π^* is one that maximizes expected utility if followed
 - An explicit policy defines a reflex (reactive/stimulus-response) agent



Policies

- Definition: a policy is a distribution over actions given states,

$$\pi(a|s) = P(A_t = a|S_t = s), \forall t$$

- A policy fully defines the behavior of an agent
- The policy is stationary (time-independent)
- During learning, the agent changes his policy as a result of experience

- Special case: deterministic policies

$$\pi(s) = \text{the action taken with prob=1 when } S_t = s$$

Markov Decision Process (MDP)

- What does Markov means?
 - Given the present state, the future and the past are independent
 - In MDPs, “Markov” means action outcomes depend only on the current state

$$\begin{aligned} P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}, \dots, S_0 = s_0) \\ = \\ P(S_{t+1} = s' | S_t = s_t, A_t = a_t) \end{aligned}$$

- This is just like search, where the successor function could only depend on the current state (not the history)
- We should be able to throw away the history once state is known

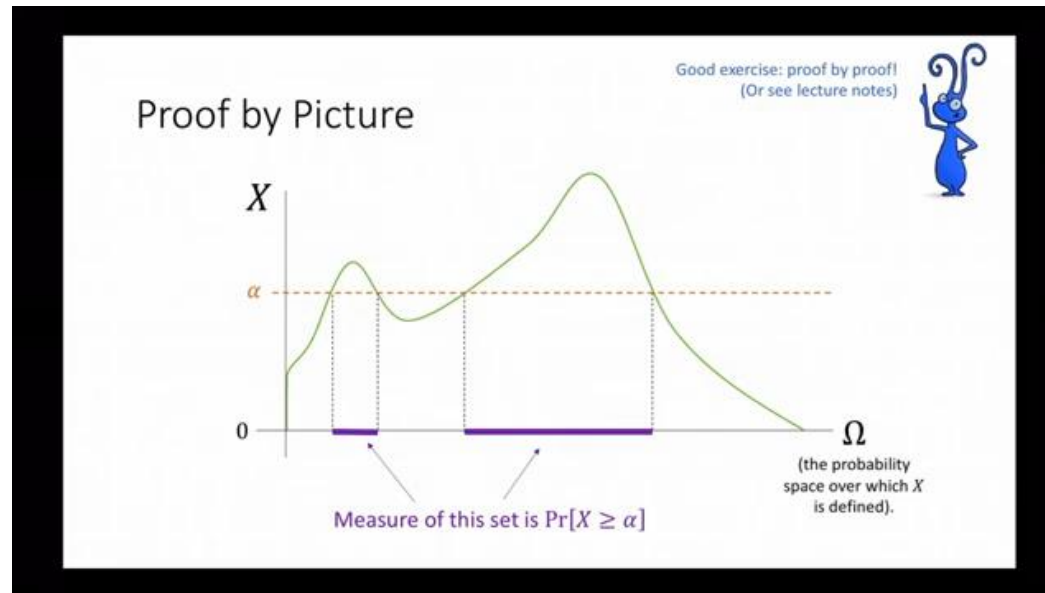


Andrey Markov
(1856-1922)

More about Markov...

- Markov Inequality: If X is a non-negative random variable, then

$$P[X \geq a] \leq \frac{\mathbb{E}[X]}{a}$$



Andrey Markov
(1856-1922)
Russian mathematician

Serves as a foundation for many other important results in probability theory (often used to prove other, tighter bounds), and it's particularly useful when we have limited information about a RV.

More about Markov...

- Chebyshev Inequality: Let X be a random variable with mean m and variance σ^2 . For any $a > 0$, $P[|X - m| \geq a] \leq \frac{\sigma^2}{a^2}$



Pafnuty Chebyshev
(1821—1894)

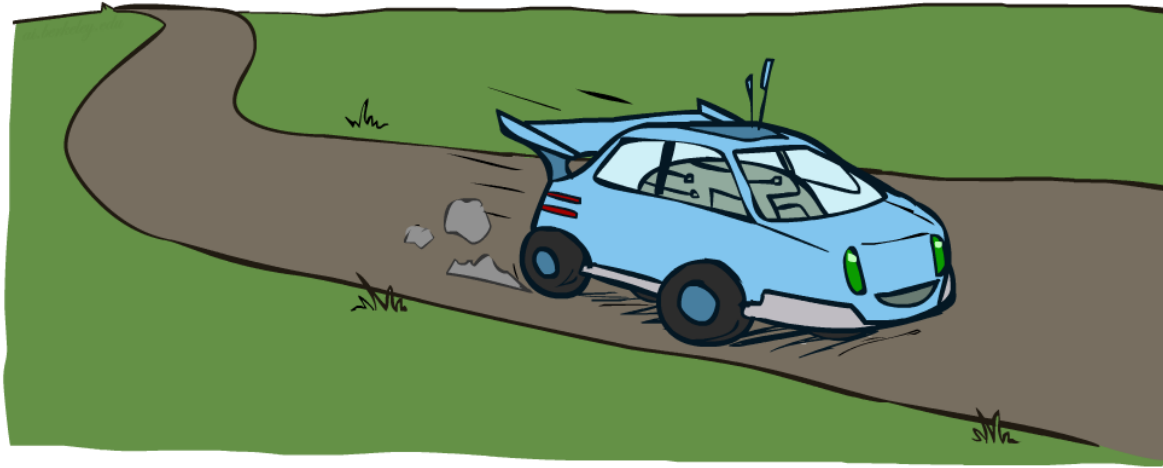
Russian mathematician
He was Markov's doctoral advisor

Fully Observable v.s. Partially Observable

- Fully observable: state is the same as observation
 - Go
- Partially Observable: only part of the states is observed
 - Robot camera
 - Poker
- If the agent is partially observable, the world is transformed from MDP into a Partially Observable MDP (POMDP).

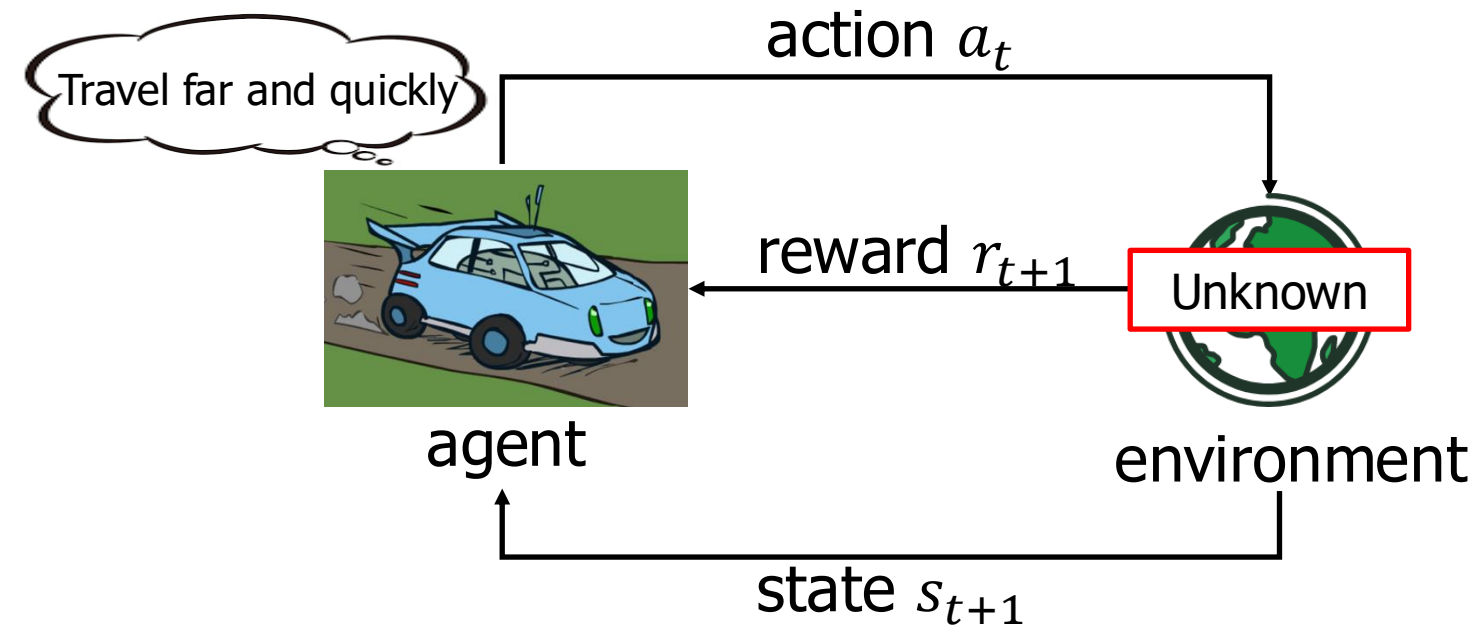
Markov Decision Process (MDP)

- Example: Racing
 - A robot car wants to travel far, quickly



Markov Decision Process (MDP)

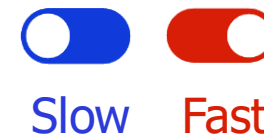
- Example: Racing
 - A robot car wants to travel far, quickly
 - Three states: Cool, Warm, Overheated
 - Two actions: Slow, Fast
 - Going faster gets double reward



- States $s \in S$



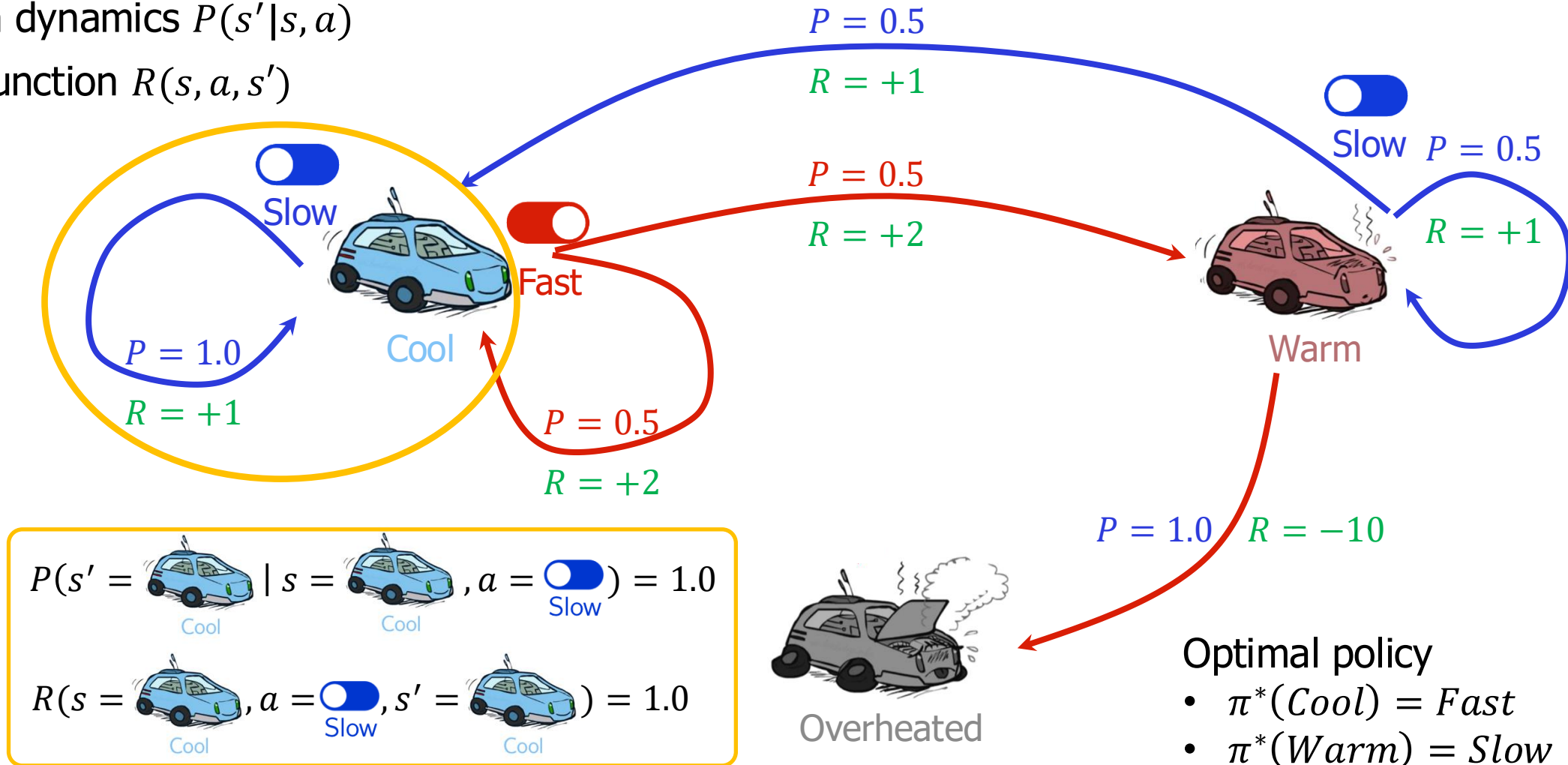
- Actions $a \in A$



Markov Decision Process (MDP)

- Example

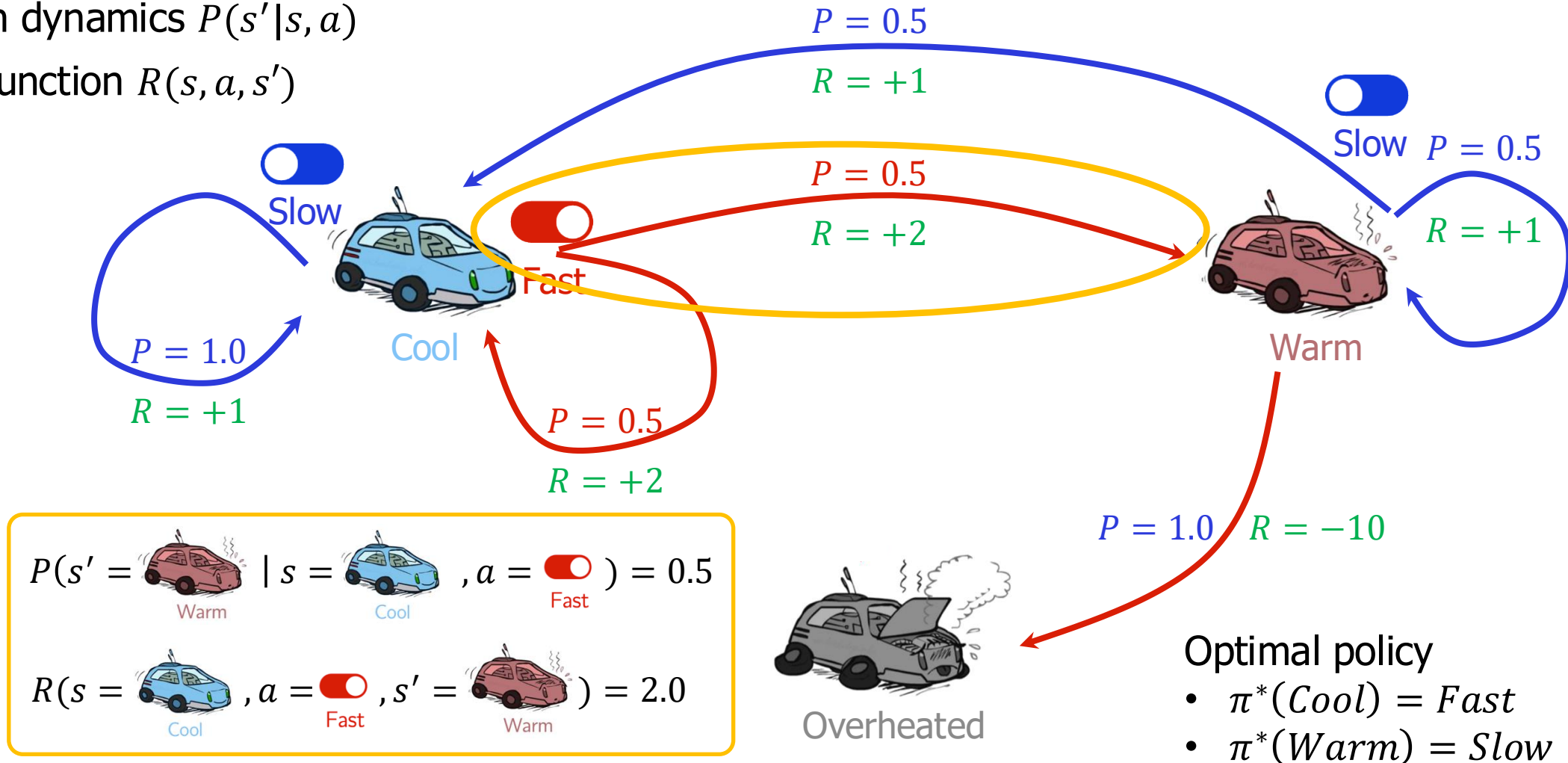
- Transition dynamics $P(s'|s, a)$
- Reward function $R(s, a, s')$



Markov Decision Process (MDP)

- Example

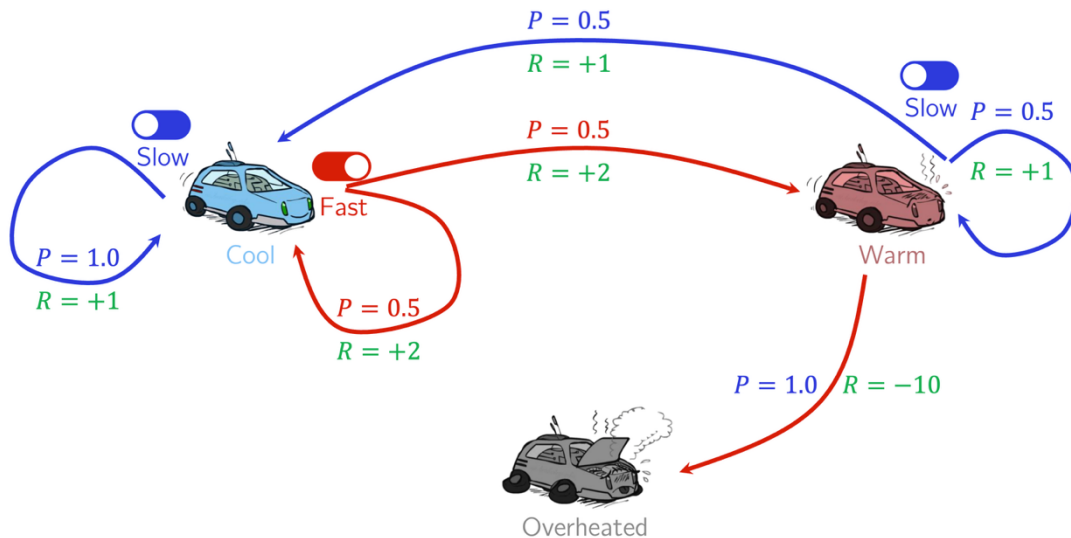
- Transition dynamics $P(s'|s, a)$
- Reward function $R(s, a, s')$



Markov Decision Process (MDP)

- Example

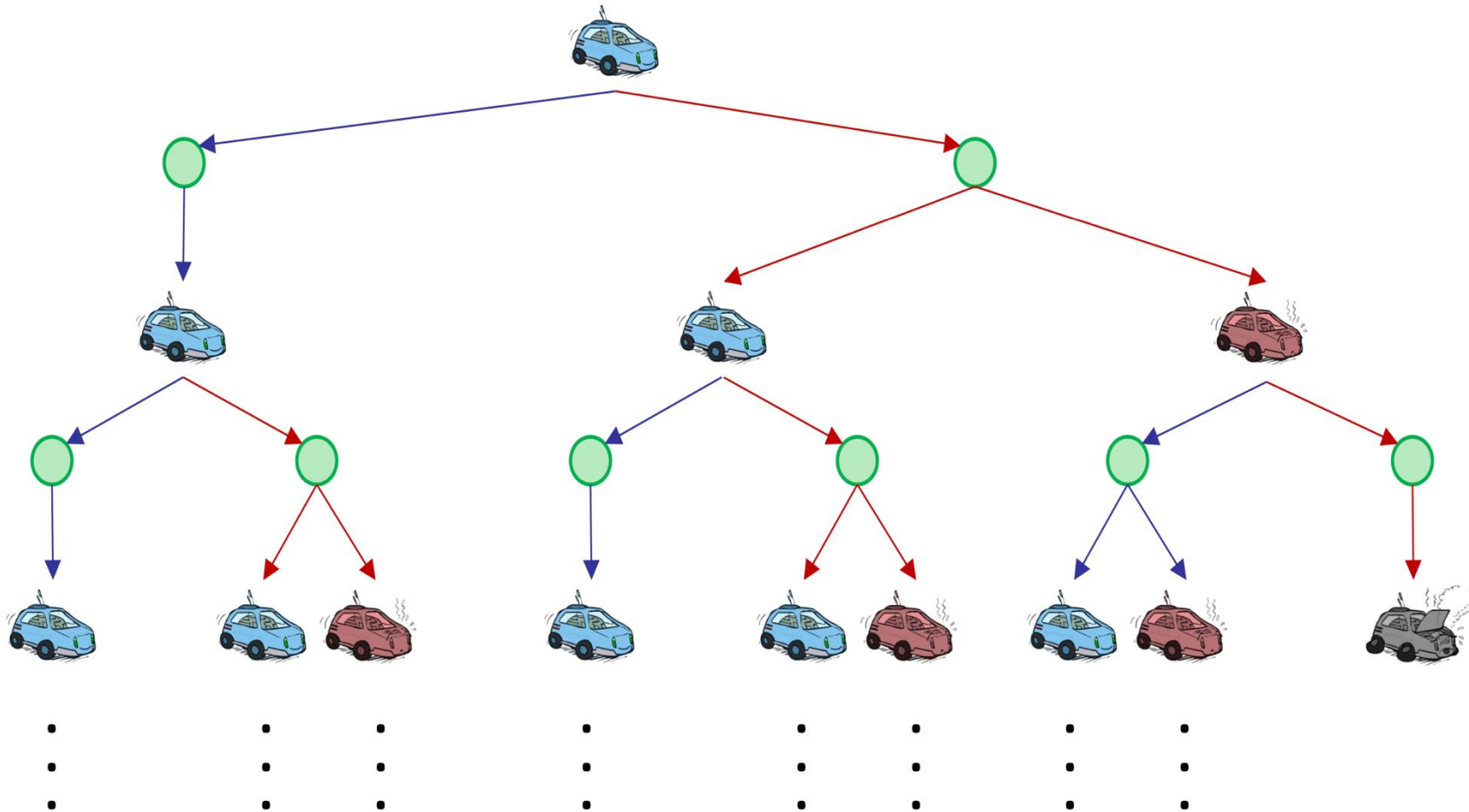
- Transition dynamics $P(s'|s, a)$
- Reward function $R(s, a, s')$



s	a	s'	$P(s' s, a)$	$R(s, a, s')$
 Cool	 Slow	 Cool	1.0	+1
 Cool	 Fast	 Cool	0.5	+2
 Cool	 Fast	 Warm	0.5	+2
 Warm	 Slow	 Cool	0.5	+1
 Warm	 Slow	 Warm	0.5	+1
 Warm	 Fast	 Overheated	1.0	-10
 Overheated	(end)	 Overheated	1.0	0

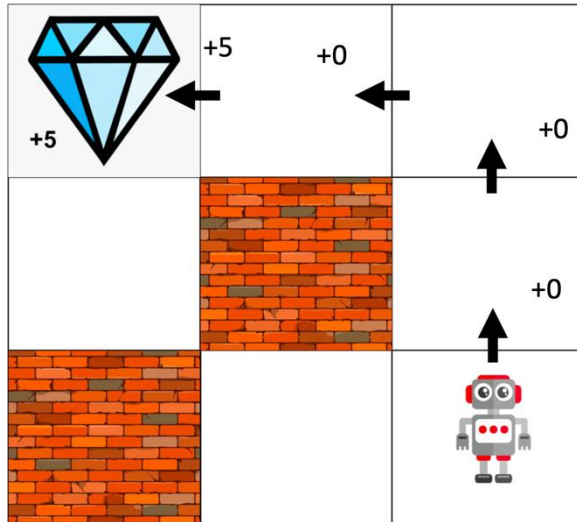
Markov Decision Process (MDP)

- Example: racing search tree



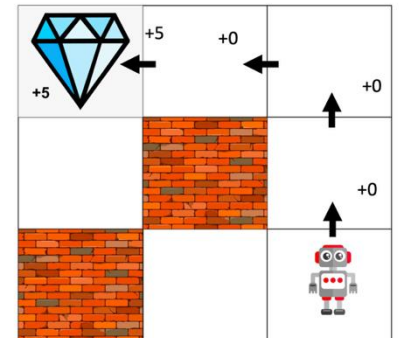
Utilities of Sequences

- Utilities are functions from outcomes (states of the world) to real numbers that describe an agent's preferences
 - Example utilities: $[0, 0, 0, 5]$



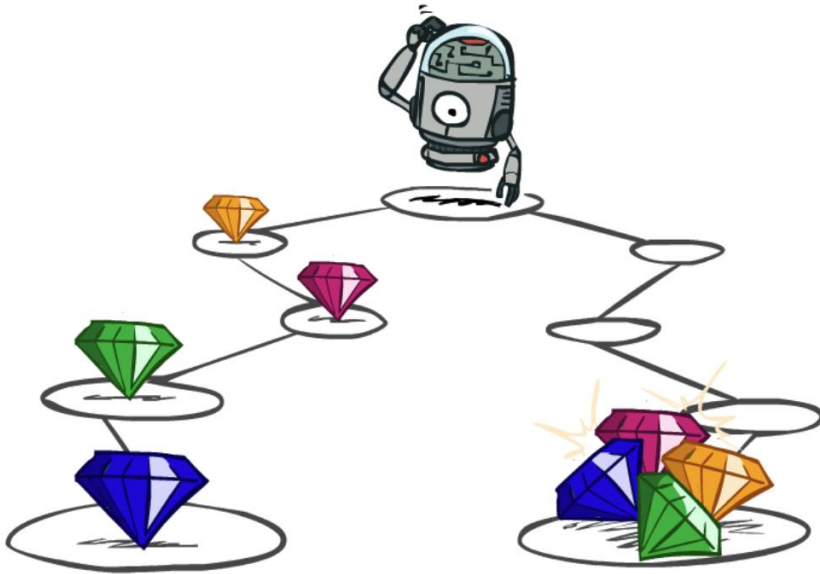
Utilities of Sequences

- Reward Hypothesis: all the goals are described by rewards
 - Rewards are scalar values provided by the environment to the agent that indicate whether goals have been achieved
 - Example: 1 if goal is achieved, 0 otherwise, or -1 for overtime step the goal is not achieved
 - Goals specify what the agent needs to achieve, not how to achieve it
 - The simplest and cheapest form of supervision: all of what we mean by goals and purposes can be well thought of as the maximization of the cumulative sum of a received scalar signal (reward):
$$r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$$
 - Goal seeking behavior, achieving purposes and expectations can be formulated mathematically as maximizing expected cumulative sum of scalar values...



Utilities of Sequences

- What preferences should an agent have over reward sequences?
 - More or less? $[1, 2, 2]$ or $[2, 3, 4]$
 - Now or later? $[0, 0, 1]$ or $[1, 0, 0]$
- What about $[1, 1, 1, 1]$ and $[0, 0, 0, 4]$?



Discounting

- It's reasonable to maximize the sum of rewards
- It's also reasonable to prefer rewards now to rewards later
- One solution: values of rewards decay exponentially (γ is between 0 and 1)



1

Worth Now



γ

Worth Next Step

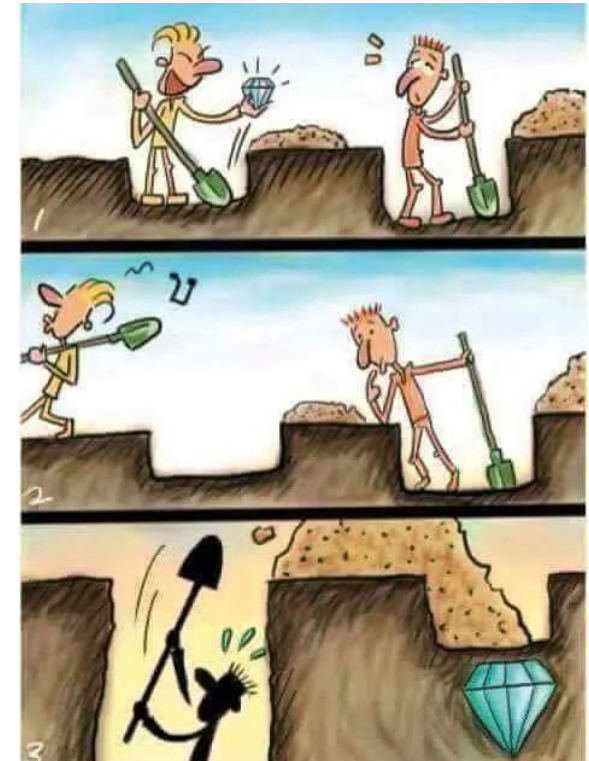
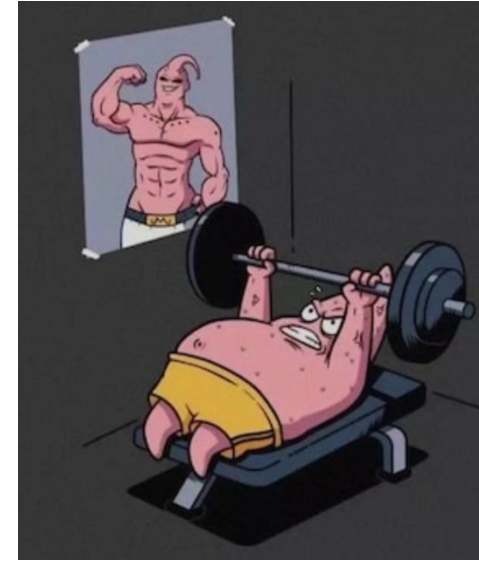


γ^2

Worth In Two Steps

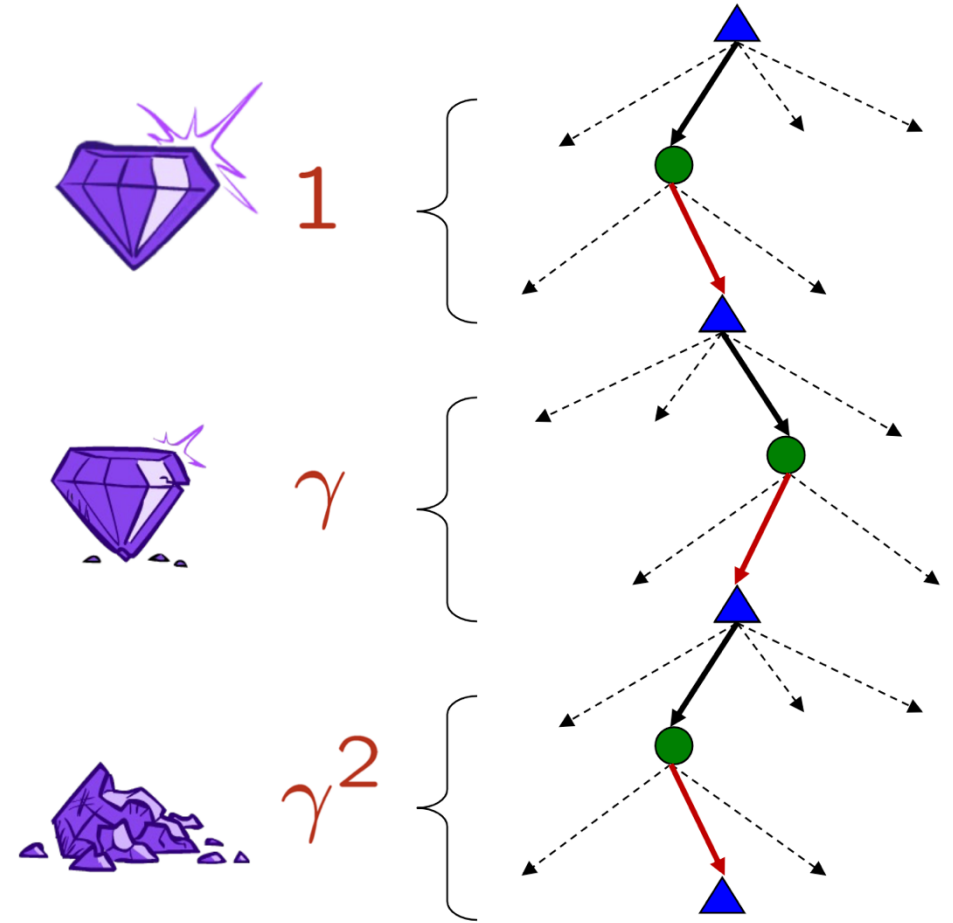
Discounting

- Why discount?
 - Intuition:
 - Sooner rewards probably do have higher utility than later rewards
 - If the reward is financial, immediate rewards may earn more interest than delayed rewards
 - Animal/human behavior shows preference for immediate reward
 - Theoretically:
 - Mathematically convenient to discount rewards
 - Avoids infinite returns in cyclic Markov processes
 - Helps our algorithms converge



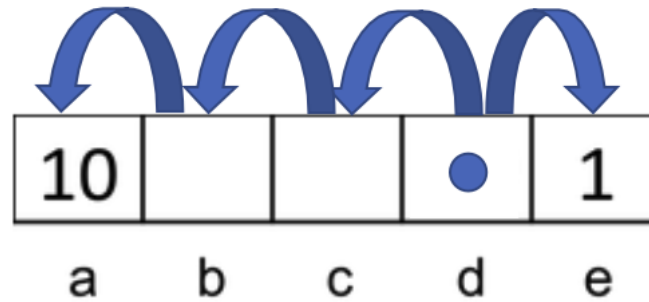
Discounting

- How to discount?
 - Each time we descend a level, we multiply in the discount once
- Example: discount of 0.5
 - $U([1,2,3]) = 1 \times 1 + 0.5 \times 2 + 0.5^2 \times 3 = 2.75$
 - $U([3,2,1]) = 1 \times 3 + 0.5 \times 2 + 0.5^2 \times 1 = 5.25$
 - $U([1,2,3]) < U([3,2,1])$



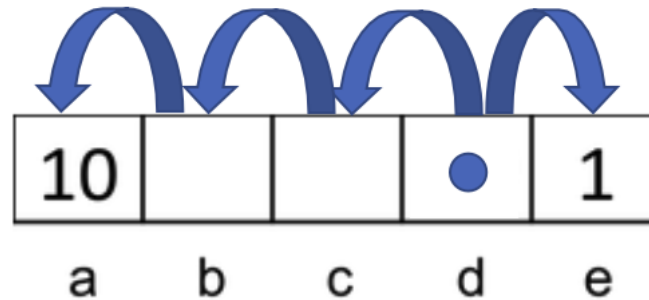
Quiz: Discounting

- Starting from d, which γ makes a and e equally good?



Quiz: Discounting

- Starting from d, which γ makes a and e equally good?



$$0 \times \gamma^0 + 0 \times \gamma^1 + 10 \times \gamma^2 = 1 \times \gamma^0 \rightarrow 10 \times \gamma^2 = 1$$

Discounted Return

- Definition: the return G_t is the total discounted reward from timestep t

$$G_T = R_{t+1} + \gamma R_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- The discount $\gamma \in [0, 1]$ determines the present value of future rewards
 - It is sometimes possible to use undiscounted Markov reward processes (i.e., $\gamma = 1$), e.g., if all sequences terminate
- The value of receiving reward R after $k + 1$ time-steps is $\gamma^k R$
- This values immediate reward above delayed reward
 - γ close to 0 leads to "myopic" evaluation
 - γ close to 1 leads to "far-sighted" evaluation

Episodic Tasks v.s. Continuing Tasks

- Episode: A sequence of interactions based on which the reward will be judged at the end
- Episodic tasks: interaction breaks naturally into episodes, e.g., plays of a game, trips through a maze
- In episodic tasks, we almost always use simple total reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

- T is a final time step at which a terminal state is reached, ending an episode

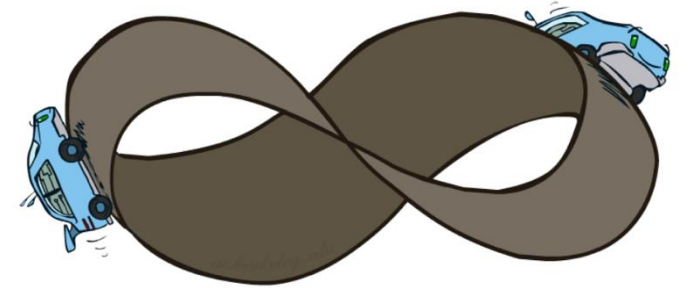
Episodic Tasks v.s. Continuing Tasks

- Continuing tasks: interaction does not have natural episodes, but just goes on and on...just like real life
- In continuing tasks, we often use simple total discounted reward:

$$G_T = R_{t+1} + \gamma R_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

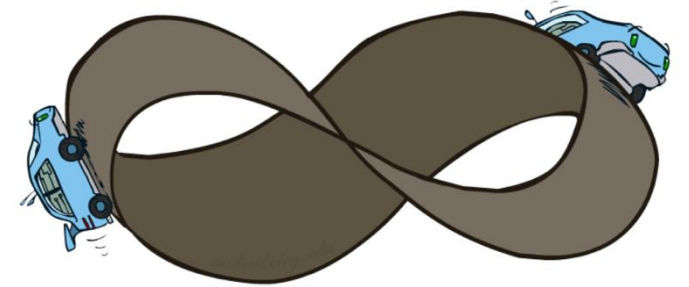
Infinite Utilities?

- Problem: What if the game lasts forever? Do we get infinite rewards?
- Solutions:
 - Finite horizon: (similar to depth-limited search)
 - Terminate episodes after a fixed T steps (e.g., life)
 - Gives nonstationary policies (π depends on time left)



Infinite Utilities?

- Problem: What if the game lasts forever? Do we get infinite rewards?
- Solutions:
 - Finite horizon: (similar to depth-limited search)
 - Terminate episodes after a fixed T steps (e.g., life)
 - Gives nonstationary policies (π depends on time left)
 - Discounting: use $0 < \gamma < 1$

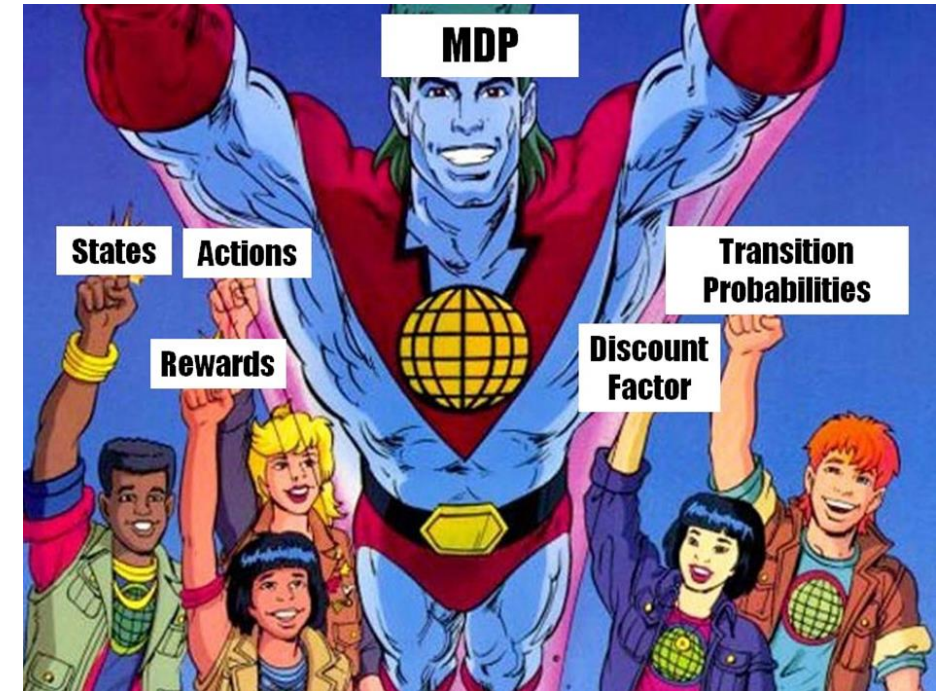


$$U([r_0, \dots, r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq R_{\max} / (1 - \gamma)$$

- Smaller γ means smaller “horizon” – shorter term focus
- Absorbing state: guarantee that for every policy, a terminal state will eventually be reached (like “overheated” for racing)

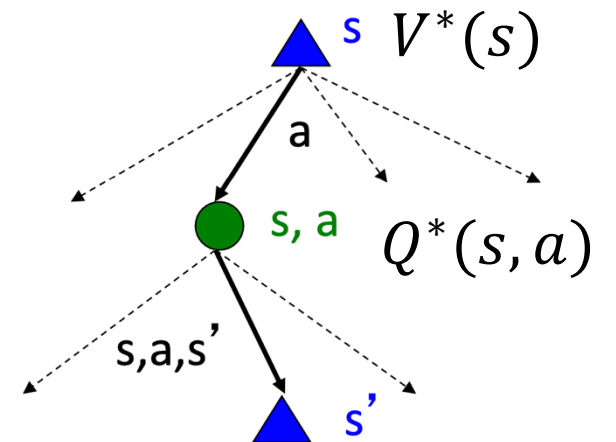
Recap: Markov Decision Process (MDP)

- An MDP is defined by:
 - A set of states $s \in S$
 - A set of actions $a \in A$
 - A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics
 - A reward function $R(s, a, s')$
 - A discount factor γ
- MDP quantities so far:
 - Policy = choice of action for each state
 - Utility = sum of discounted rewards



Optimal Quantities

- The value (utility) of a state s :
 - $V^*(s)$ = expected utility starting in s and act optimally
 - The optimal state-value function $V^*(s)$ is the maximum value function over all policies $V^*(s) = \max_{\pi} V^{\pi}(s)$
- The value (utility) of a q-state (s, a) :
 - $Q^*(s, a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally
 - The optimal action-value function $Q^*(s, a)$ is the maximum action-value function over all policies $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$
- The optimal policy:
 - $\pi^*(s) =$ optimal action from state s



Solving MDPs

- Prediction:
 - Given an MDP (S, A, T, r, γ) and a policy $\pi(a|s) = P[A_t = a|S_t = s]$, find the state and action value functions
- Optimal control:
 - Given an MDP (S, A, T, r, γ) , find the optimal policy (the planning problem).
 - Compare with the learning problem where rewards and dynamics are unknown.

Quantities

- Value functions:
 - Measure the goodness of a particular state or state/ action pair
 - How good is for the agent to be in a particular state or execute a particular action at a particular state, for a given policy?
- Optimal value functions:
 - Measure the best possible goodness of states or state/action pairs under all possible policies

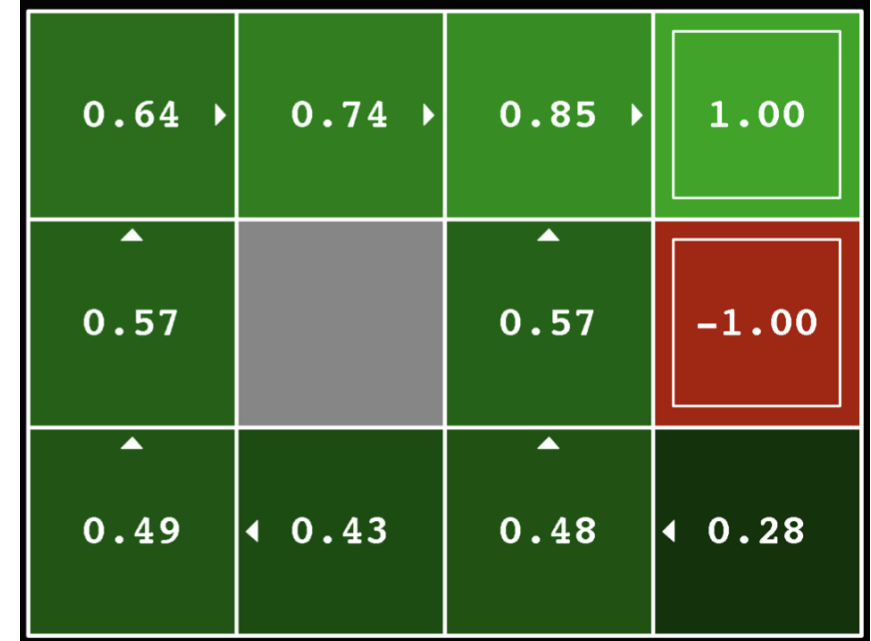
	state values	action values
prediction	V_{π}	q_{π}
control	V_{*}	q_{*}

Quantities

- Why value functions are useful?
 - Value functions capture the knowledge of the agent regarding how good is each state for the goal she is trying to achieve.
“... knowledge is represented as a large number of approximate value functions learned in parallel ...”

Optimal Quantities

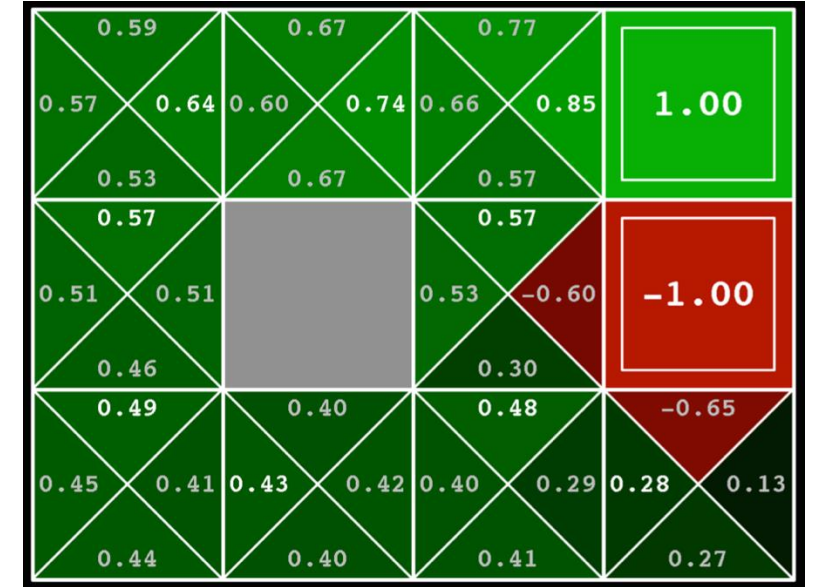
- Example: Gridworld
 - The value (utility) of a state s :
 - $V^*(s)$ = expected utility starting in s and act optimally
 - The value (utility) of a q-state (s, a) :
 - $Q^*(s, a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally
 - The optimal policy:
 - $\pi^*(s)$ = optimal action from state s



Noise = 0.2
Discount = 0.9
Living reward = 0

Optimal Quantities

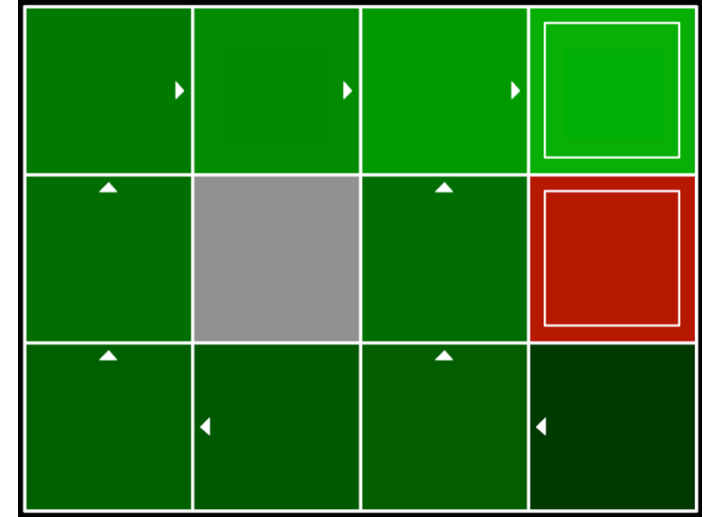
- Example: Gridworld
 - The value (utility) of a state s :
 - $V^*(s)$ = expected utility starting in s and act optimally
 - The value (utility) of a q-state (s, a) :
 - $Q^*(s, a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally
 - The optimal policy:
 - $\pi^*(s)$ = optimal action from state s



Noise = 0.2
Discount = 0.9
Living reward = 0

Optimal Quantities

- Example: Gridworld
 - The value (utility) of a state s :
 - $V^*(s)$ = expected utility starting in s and act optimally
 - The value (utility) of a q-state (s, a) :
 - $Q^*(s, a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally
 - The optimal policy:
 - $\pi^*(s)$ = optimal action from state s



Noise = 0.2
Discount = 0.9
Living reward = 0

Value of States

- Recursive definition of value (Bellman equation)

- Optimal state value function:

$$V^*(s) = \max_a Q^*(s, a)$$

- Example: $V^*(\cdot) = \max_a \{Q^*(\cdot, \uparrow), Q^*(\cdot, \downarrow), Q^*(\cdot, \leftarrow), Q^*(\cdot, \rightarrow)\}$

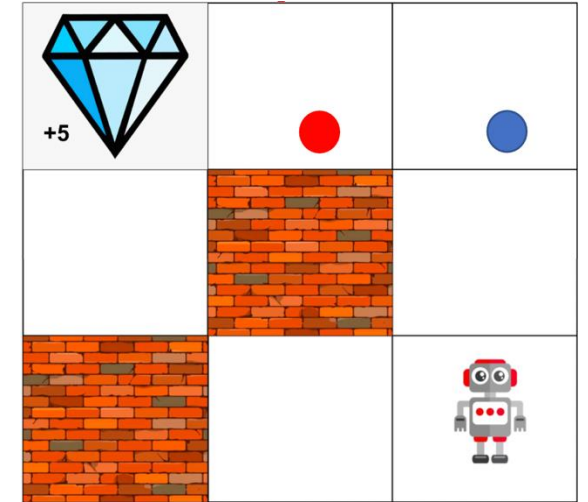
- Optimal state-action value function:

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- Therefore:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

How do we solve these equations?



Value of States

- Why value functions are useful?

- An optimal policy can be found by maximizing over $Q^*(s, a)$:

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \operatorname{argmax}_a Q^*(s, a) \\ 0, & \text{otherwise} \end{cases}$$

- An optimal policy can be found from the model dynamics using one step look ahead:

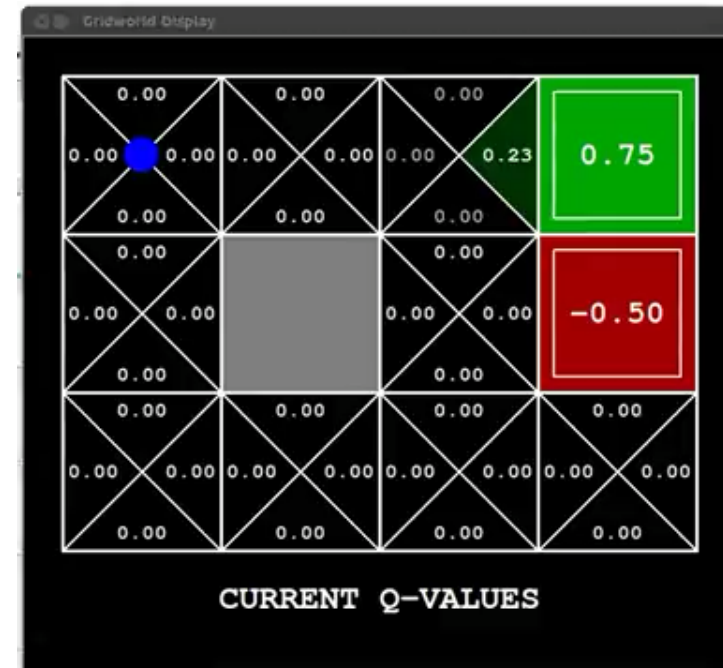
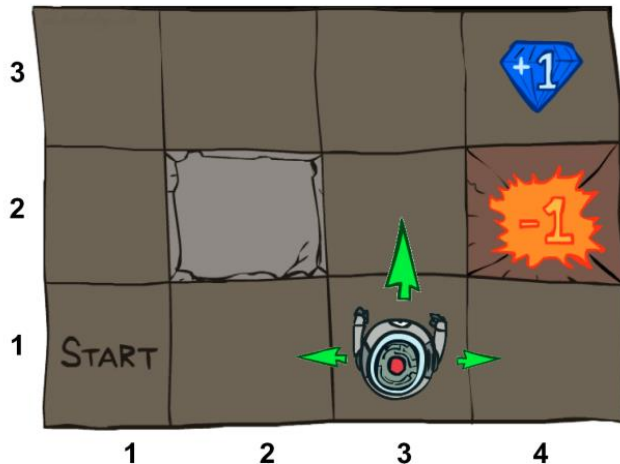
$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')] \\ 0, & \text{otherwise} \end{cases}$$

- If we know $Q^*(s, a)$: we immediately have the optimal policy, we do not need the dynamics!
 - If we know $V^*(s)$, we need the dynamics to do one step lookahead, to choose the optimal action

Problem: How to solve these equations?

- $V^*(s) = \max_a Q^*(s, a)$
- $Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$

- Next lecture: Solving MDPs



Thanks!